本科生毕业论文（设计）

题目：PINN 求解半经典薛定谔方程的相关研究以及分析

姓　　名　龚立言

院　　系　数学学院

专　　业　数学与应用数学

指导教师　柴利慧教授

2026 年 10 月 22 日

PINN 求解半经典薛定谔方程的相关研究以及分析

PINN method for semi-classsical Schrödinger equation and related analysis

姓　　名　龚立言

院　　系　数学学院

专　　业　数学与应用数学

指导教师　柴利慧教授

2026 年 10 月 22 日

学术诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文（设计）不包含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本论文（设计）的知识产权归属于培养单位。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日　　期：　　年　　月　　日

摘要

本文研究利用物理信息神经网络（PINN）数值求解半经典薛定谔方程（Semi-classical Schrödinger Equation）的方法。传统数值方法在处理高维或复杂边界条件时计算成本较高，而 PINN 通过结合神经网络和物理方程约束，提供了一种高效的解决方案。本文重点研究了 PINN 在半经典薛定谔方程中的应用，包括如何利用 PINN 得到 WKB 近似解，以及 PINN 在求解半经典薛定谔方程优化采样策略以提高计算精度和效率。数值实验表明，该方法能够有效求解半经典薛定谔方程等相关问题，并保持较高的计算精度。此外，我还分析了不同网络结构和训练策略对结果的影响，并讨论了该方法在更复杂情况下的应用潜力。

关键词：半经典薛定谔方程, 物理信息神经网络 (PINN), WKB 近似, 数值计算, 深度学习

ABSTRACT

This paper investigates the use of Physics-Informed Neural Networks (PINNs) for numerically solving the Semi-classical Schrödinger Equation. Traditional numerical methods often incur high computational costs when dealing with high-dimensional problems or complex boundary conditions. In contrast, PINNs provide an efficient solution by integrating neural networks with physical equation constraints. This study focuses on the application of PINNs to the Semi-classical Schrödinger Equation, including how PINNs can be utilized to obtain WKB approximate solutions and how optimized sampling strategies can enhance computational accuracy and efficiency. Numerical experiments demonstrate that this method effectively solves the Semi-classical Schrödinger Equation and related problems while maintaining high computational accuracy. Additionally, I analyze the impact of different network architectures and training strategies on the results and discuss the potential applications of this approach in more complex scenarios.

Keywords: Semi-classical Schrödinger Equation, Physical Information Neural Networks, WKB approximation, Numerical computation, Deep learning

目录

1 绪论 1
1.1 研究背景 1
1.2 研究意义 1
1.3 文献综述 2
1.4 研究方法和框架 4
2 理论推导 5
2.1 半经典薛定谔方程 (Semi-classical Schrödinger Equation) 5
2.2 传统数值方法 5
2.3 WKB 近似 (WKB approximation)[22] 10
2.4 物理信息神经网络 (Physics-Informed Neural Networks, PINN) 12
2.5 使用 PINN 去获得 WKB 解 17
2.6 基于物理信息神经网络的 WKB 近似求解方法 17
3 采点分析 27
3.1 想法 27
3.2 取点策略 27
参考文献 32
附录 A WKB 求解代码 34
附录 B 常规采样分析代码 55
致谢 60

1 绪论

1.1 研究背景

1.1.1 薛定谔方程

薛定谔方程（Schrödinger Equation）是量子力学的核心方程，描述了量子系统随时间的演化。薛定谔根据路易 · 德布罗意（Louis de Broglie）的假设提出了这一方程，德布罗意假设所有物质都具有与之相关的物质波。从物理意义上讲，薛定谔方程是经典力学中牛顿第二定律的量子版本。在已知初始条件的情况下，牛顿第二定律可以帮助我们预测一个物理系统随时间演化的轨迹。而薛定谔方程则描述了波函数（wave function）随时间的变化，波函数则是孤立物理系统在量子力学中的表现。

其数学形式是一类偏微分方程（PDEs），通常具有以下两种形式：时间依赖薛定谔方程（Time-dependent Schrödinger Equation, TDSE）：

$$i\hbar\frac{\partial}{\partial t}\psi(\mathbf{r},t)=\hat{H}\psi(\mathbf{r},t)$$

其中，$\psi(\mathbf{r},t)$ 是波函数，$\hat{H}$ 是哈密顿算符，$\hbar$ 是约化普朗克常数。以及时间无关薛定谔方程（Time-independent Schrödinger Equation, TISE）：

$$\hat{H}\psi(\mathbf{r})=E\psi(\mathbf{r})$$

其中 E 是体系的能量本征值，该方程用于求解稳态量子系统的能级与波函数。

1.2 研究意义

研究薛定谔方程具有重要意义：通过求解不同势能下的波函数，我们可以预测粒子的可能轨迹、散射行为、束缚态等；量子计算的基本单元——量子比特（Qubit）[1]可用薛定谔方程描述其演化；在研究超导[2]、玻色-爱因斯坦凝聚（BEC）[3]等现象时，薛定谔方程的非线性推广（如 Gross-Pitaevskii 方程）[4]的重要工具；在量子热化、开放量子系统等领域[5]，研究非平衡系统的时间演化[6]对于理解热力学第二定律的微观机制也具有重要意义。甚至在化学与分子动力学、材料科学、天体物理等多个领域，薛定谔方程也发挥着重要作用。

1.3 文献综述

薛定谔方程作为量子力学的核心基本方程，自其提出以来一直是数学物理领域的重要研究对象。该方程在描述微观粒子运动规律方面具有不可替代的理论价值，其求解问题对于理解量子系统的动力学行为[1] [2] [3] [4] [5]具有基础性意义。从数学角度而言，薛定谔方程的求解涉及到偏微分方程理论、泛函分析等现代数学工具的应用；从物理角度来看，其解析解或近似解能够为量子系统的能级结构、波函数分布[6]等关键物理特性提供直接的理论依据。

本文所要研究的半经典薛定谔方程作为一类特殊的薛定谔方程形式，在联系经典力学与量子力学方面发挥着桥梁作用。这类方程通常出现在量子系统具有明显半经典特征的物理情境中，其求解不仅具有理论上的挑战性，更在图像识别[7]、基因组学[8]，认知科学[9]等实际应用中展现出重要的实用价值。针对其半经典特性的数学分析，Carles[10]建立了非线性薛定谔方程的半经典分析框架，通过 WKB 近似和几何光学方法揭示了高频振荡解的传播机制。正因如此，半个多世纪以来，众多研究者在传统数值方法的基础上[11]持[12]续[13] [14]致力于发展各类解析和数值方法来求解半经典薛定谔方程，形成了丰富的理论体系和方法学积累。Bao 等学者[15]通过时间分裂谱方法实现了半经典极限下非线性薛定谔方程的高效求解，其数值实验表明该方法在 $\epsilon \to 0$ 时仍能保持 $\mathbb{L}^2$ 范数守恒性。本文所研究的半经典薛定谔方程求解问题，正是基于前人的理论研究和方法框架，进行的扩展和优化。

物理信息神经网络（Physics-Informed Neural Networks, PINNs）作为融合深度学习与物理建模的新型计算范式，近年来在偏微分方程求解领域展现出独特优势。Raissi 等学者于 2019 年发表的文章[16]提出了 PINN 的理论框架，通过将物理守恒定律以偏微分方程的形式嵌入神经网络的正则化项，构建了兼具数据驱动与物理约束的深度学习模型。该方法的核心技术依赖于自动微分（Automatic Differentiation）[17]，使得神经网络能够精确计算偏微分方程中的高阶导数项。该方法突破了传统数值方法对网格离散化的依赖，在处理高维问题时具有显著的计算效率优势。近期研究进一步表明，配置点采样策略对 PINN 性能具有关键影响，Wu 等[18]系统分析了非自适应与残差自适应采样的优化效果，发现自适应方法可将收敛速度提升百分之 40 以上。

同时，Raissi 及其合作者率先将 PINN 应用于非线性薛定谔方程的求解，成功实现了波函数的高精度求解。相较于有限差分法等传统数值方法，PINN 无需预先网格化，而是通过自适应采样配置点进行无网格求解，这种特性使其在复杂几何域和高维扩展问题中更具优势。文章进一步提出了连续时间与离散时间两类算法

模型：前者通过构建物理信息正则化项实现时空连续建模，后者结合隐式龙格-库塔方法提高了时间积分精度。数值实验表明，该方法在浅水波传播、反应扩散系统等典型问题上均展现出良好的泛化能力。

然而，PINN 方法仍存在一定缺陷。首先，模型精度对配置点数量和分布具有较强依赖性[18] [19]，在处理强非线性或奇异问题时需要大量采样点，导致计算成本显著增加。其次，尽管实证研究验证了方法的有效性，但关于网络收敛性、逼近误差估计等数学理论尚未有严格证明。此外，文章也指出在小数据场景下，相较于高斯过程回归等概率方法，确定性神经网络可能面临过拟合风险。Brevi 等[20]在量子系统谱计算中提出混合方法，将 PINN 与谱元法结合，成功降低了高频振荡导致的相位误差。该文章为计算数学提供了新工具，展了基于深度学习的 PDE 数值解法。

回到本文着重研究的半经典薛定谔方程，针对其的高效数值求解问题，柴利慧等学者[21]发展了周期性势场下的半经典模型，通过带间跃迁处理解决了能带交叉引起的数值不稳定性问题。金石在 2011 年的文章[22]系统系统性阐述了当前用于研究薛定谔型高振荡偏微分方程（PDEs）的数值方法，特别聚焦于高频振荡解的数值处理策略。文章指出，此类方程因半经典参数 $\varepsilon \ll 1$ 引发的时空多尺度特性，导致常规数值方法面临严重的计算不稳定性：其中，有限差分法与有限元法在 $\varepsilon \to 0$ 时需网格尺寸 $\Delta(x) = o(\varepsilon)$ 的严格限制，使得计算复杂度呈指数级增长。在此背景下，谱方法展现出独特优势，Driscoll 等开发的 Chebfun 工具包[23]通过谱配置法实现了高精度函数逼近，为半经典问题求解提供了新范式。为此，作者重点探讨了 WKB 近似——通过将振幅和相位函数分开成 $u^\varepsilon(t,x) = a^\varepsilon(t,x)e^{iS(t,x)/\varepsilon}$，有效降低了高频振荡对离散格式的敏感性。

在收敛性分析层面，该研究揭示了传统稳定性-相容性框架的局限性：即便采用隐式龙格-库塔等高阶时间积分方案，固定 ε 下的局部截断误差估计仍无法保证半经典极限过程的均匀收敛性。为此，作者还提出将微局部分析工具（如 Wigner 测度理论）引入数值格式设计，通过相空间能量输运方程构建 ε 一致稳定的离散格式。Iserles 的数值分析专著[24]为此提供了理论支撑，其关于刚性方程和几何积分的讨论为多尺度问题的时间离散策略设计提供了重要参考。这类相空间方法虽能规避焦散现象并保持 L^2 范数守恒，却以丢失波函数相位信息为代价，限制了其在非线性问题中的直接应用。对此，文章进一步将它和时间分裂下的谱方法进行对比：该谱方法通过 Fourier 基函数的全局逼近特性，将计算复杂度从 $O(\varepsilon^{-d-1})$ 降至 $O(|log\varepsilon|^d)$，更适用于高维周期场景。

金石老师的文章为我们后续的研究提供了重要的启示：首先，我们可以根据

WKB 求解出某些特定情况下，半经典薛定谔方程的近似解；其次，我们应当根据势函数 $V(x)$ 的数学特性, 如连续性、周期性、随机性等，动态选择离散策略；最后，非线性项的存在会通过自相互作用产生新的振荡尺度，这要求数值格式具备多分辨率特征。这些洞见为本文研究半经典薛定谔方程的数值求解以及建模奠定了理论基础。

1.4 研究方法和框架

本文首先综述了半经典薛定谔方程的研究背景，系统分析了传统数值方法的局限性，并介绍了 WKB 近似与物理信息神经网络（PINN）的基本原理。同时表明，PINN 在求解薛定谔方程时较传统方法具有显著优势。随后，本文详细阐述了 PINN 求解薛定谔方程的具体实现方法，并针对提高计算效率的关键环节——采样策略进行了优化与理论分析。

2 理论推导

在已经提出 PINN 去求解薛定谔方程的前提下，我们希望研究相关的 WKB 解和如何通过寻找一个好的采样方式去提升求解精度.

2.1 半经典薛定谔方程 (Semi-classical Schrödinger Equation)

定义 2.1 (半经典薛定谔方程)

$$i\varepsilon\partial_t u^\varepsilon = -\frac{\varepsilon^2}{2}\Delta u^\varepsilon + V(x)u^\varepsilon, u^\varepsilon(0,x) = u^\varepsilon_{in}(0,x)$$

其中，u^ε 一般表示波函数，$V(x)$ 表示势能函数，$(t,x) \in \mathbb{R}\times\mathbb{R}^d$, $d \in N$ 表示相空间 (space dimension). 此外, $\varepsilon \in (0,1]$ 表示半经典参数，即归一化后的普朗克常数 (the scaled Planck's constant), 描述了它们的微观/宏观尺寸.

2.2 传统数值方法

为了求解薛定谔方程，以下有一些传统数值方法可以使用：

2.2.1 有限差分法 (FDM)[11]

定义：最常见的数值方法，将微分算子离散化为差分形式，将薛定谔方程转化为线性代数方程组。这里以半经典薛定谔薛定谔方程为例：

$$i\varepsilon\partial_t u^\varepsilon(t,x) = -\frac{\varepsilon^2}{2}\Delta u^\varepsilon(t,x) + V(x)u^\varepsilon(t,x) \tag{2.1}$$

如果采用 **Crank-Nicolson 格式**, 那么简单分为以下四步:

1. **时间离散**：在 t_n 到 t_{n+1} 区间取梯形积分

$$i\varepsilon\frac{u^{n+1}-u^n}{\Delta t} = -\frac{\varepsilon^2}{4}(\partial_{xx}u^{n+1} + \partial_{xx}u^n) + \frac{V}{2}(u^{n+1}+u^n) \tag{2.2}$$

2. **空间离散**：采用中心差分 δ_x^2

$$\frac{\varepsilon^2}{2}\frac{u_{j+1}-2u_j+u_{j-1}}{(\Delta x)^2} \approx \varepsilon^2\partial_{xx}u \tag{2.3}$$

3. **全离散格式**：

$$i\varepsilon\frac{u_j^{n+1}-u_j^n}{\Delta t} = -\frac{\varepsilon^2}{4}\left(\frac{u_{j+1}^{n+1}-2u_j^{n+1}+u_{j-1}^{n+1}}{(\Delta x)^2}\frac{u_{j+1}^{n}-2u_j^{n}+u_{j-1}^{n}}{(\Delta x)^2}\right) + \frac{V_j}{2}\left(u_j^{n+1}+u_j^n\right) \tag{2.4}$$

4. **矩阵形式重构**

整理得到线性系统：

$$(\mathbf{I} + i\beta\mathbf{A})\mathbf{u}^{n+1} = (\mathbf{I} - i\beta\mathbf{A})\mathbf{u}^{n} \tag{2.5}$$

其中：

$$\beta = \frac{\varepsilon \Delta t}{4(\Delta x)^2} \tag{2.6}$$

$$\mathbf{A} = \begin{bmatrix} 2+\gamma_1 & -1 & & \\ -1 & \ddots & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2+\gamma_M \end{bmatrix}, \quad \gamma_j = \frac{2(\Delta x)^2 V_j}{\varepsilon^2} \tag{2.7}$$

优势：

- 实现简单，适用于一维/二维等低维问题
- 可直观处理规则区域和均匀网格

劣势：

- 在处理高维问题时，计算量呈指数增长，不适用于处理高维薛定谔方程问题
- 边界条件对方程的影响很大，同时非常依赖网格，网格精度影响稳定性

2.2.2 变分法[12]

定义：同样以半经典薛定谔方程为例：

$$\hat{H}\psi = E\psi, \hat{H} = -\frac{\varepsilon^2}{2}\frac{d^2}{dx^2} + V(x), E(x) = i\varepsilon\frac{d}{dt} \tag{2.8}$$

基于变分原理，选取试探波函数 $\psi_{\text{trial}}(\boldsymbol{\theta})$，且最小化能量泛函如下：

$$E_{\text{trial}} = \frac{\langle \psi_{\text{trial}} | \hat{H} | \psi_{\text{trial}} \rangle}{\langle \psi_{\text{trial}} | \psi_{\text{trial}} \rangle} \tag{2.9}$$

通过优化参数 $\boldsymbol{\theta}$ 逼近基态能量。

数学原理　根据量子力学的 Rayleigh-Ritz 定理[25]，对于任意试探波函数 ψ_{trial}，其能量期望值满足：

$$E_0 \leq \frac{\psi_{\text{trial}} \hat{H} \psi_{\text{trial}}}{\psi_{\text{trial}} \psi_{\text{trial}}} \tag{2.10}$$

其中 E_0 为真实基态能量，$\hat{H}$ 为哈密顿算符。所以其能量期望值 $E(\alpha)$ 总是大于或等于真实的基态能量 E_0，因此，我们可以不断最小化 $E(\alpha)$ 来逼近 E_0

1. 选择试探波函数　首先选择一个形式简单但包含可调参数的试探波函数 $\psi_{\text{trial}}(x, \alpha)$，其中 α 是可调的参数集。试探波函数可以根据问题的对称性或者经典物理理解来选择

2. 计算能量期望值 能量期望值 $E(\alpha)$ 对应于哈密顿量 $\hat{H}$ 在试探波函数下的期望值。假设哈密顿量为：

$$\hat{H} = -\frac{\varepsilon^2}{2}\frac{d^2}{dx^2} + V(x)$$

则能量期望值 $E(\alpha)$ 计算为：

$$E(\alpha) = \frac{\langle \psi_{\text{trial}}(\alpha)|\hat{H}|\psi_{\text{trial}}(\alpha)\rangle}{\langle \psi_{\text{trial}}(\alpha)|\psi_{\text{trial}}(\alpha)\rangle}$$

这个表达式分为两个部分：

$$E(\alpha) = (\underbrace{T(\alpha)}_{\text{动能项}} + \underbrace{V(\alpha)}_{\text{势能项}}) \cdot \frac{1}{\text{归一化常数}} \tag{2.11}$$

其中，动能项：

$$T(\alpha) = \langle \psi_{\text{trial}}(\alpha)| -\frac{\varepsilon^2}{2}\frac{d^2}{dx^2}|\psi_{\text{trial}}(\alpha)\rangle$$

势能项是：

$$V(\alpha) = \langle \psi_{\text{trial}}(\alpha)|V(x)|\psi_{\text{trial}}(\alpha)\rangle$$

这个期望值可以通过直接积分计算，其中 $V(x)$ 是势能函数。归一化常数计算为：

$$\langle \psi_{\text{trial}}(\alpha)|\psi_{\text{trial}}(\alpha)\rangle = \int_{-\infty}^{\infty} |\psi_{\text{trial}}(x,\alpha)|^2\, dx$$

对于以上三项，我们均可以通过数值积分进行计算

3. 最小化能量期望值 一旦我们得到了能量期望值 $E(\alpha)$，接下来需要最小化它来逼近真实基态能量 E_0。我们可以通过数值方法（如梯度下降、牛顿法等）找到使得 $E(\alpha)$ 最小的 α_{opt}。通过优化过程得到的参数 α_{opt} 对应的试探波函数 $\psi_{\text{trial}}(x,\alpha_{\text{opt}})$ 就是我们近似得到的基态波函数, 也就是波函数的最佳近似值, 此时得到的能量 $E(\alpha_{\text{opt}})$ 就是我们通过变分法得到的基态能量的最佳近似值。

优势：

- 对于基态和低激发态能量可以做到高效计算
- 更方便结合物理方面的内容设计试探波函数

劣势：

- 结果严重依赖试探波函数的选择，如果在物理学中没得到解释，试探函数的选择可以说是大海捞针
- 很难处理高激发态的情形

2.2.3 时间依赖有限元方法 (TD-FEM)[13]

1. **区域离散化**：将空间求解域 Ω 划分为有限个单元（如三角形/四面体），保持与稳态 FEM 相同的空间离散策略
2. **时空基函数构造**：在空间基函数 $\phi_j(\mathbf{r})$ 基础上引入时间变量，将波函数表示为时空分离形式：

$$\psi(\mathbf{r},t) \approx \sum_{j=1}^{N} c_j(t)\phi_j(\mathbf{r}) \tag{2.12}$$

   其中 $c_j(t)$ 为随时间演化的系数
3. **建立弱形式**：将时间依赖薛定谔方程

$$i\varepsilon\frac{\partial\psi}{\partial t} = \left(-\frac{\varepsilon^2}{2}\nabla^2 + V\right)\psi$$

   转化为 Galerkin 弱形式：

$$i\varepsilon\int_\Omega \phi_i\frac{\partial\psi}{\partial t}d\mathbf{r} = \int_\Omega\left[\frac{\varepsilon^2}{2}\nabla\phi_i\cdot\nabla\psi + \phi_i V\psi\right]d\mathbf{r} \tag{2.13}$$

4. **半离散化系统**：代入基函数展开式，得到常微分方程组 (ODE)：

$$i\varepsilon\mathbf{M}\frac{d\mathbf{c}}{dt} = \mathbf{K}\mathbf{c} \tag{2.14}$$

   其中 $\mathbf{M}_{ij} = \langle\phi_i|\phi_j\rangle$ 为质量矩阵，$\mathbf{K}_{ij} = \langle\nabla\phi_i|\frac{\varepsilon^2}{2}|\nabla\phi_j\rangle + \langle\phi_i|V|\phi_j\rangle$ 为刚度矩阵
5. **时间离散化**：采用 Crank-Nicolson 格式进行时间积分，在 t^n 到 t^{n+1} 时间步内：

$$i\varepsilon\mathbf{M}\frac{\mathbf{c}^{n+1}-\mathbf{c}^n}{\Delta t} = \frac{1}{2}\mathbf{K}\left(\mathbf{c}^{n+1}+\mathbf{c}^n\right) \tag{2.15}$$

   整理得线性系统：

$$\left(i\varepsilon\mathbf{M} - \frac{\Delta t}{2}\mathbf{K}\right)\mathbf{c}^{n+1} = \left(i\varepsilon\mathbf{M} + \frac{\Delta t}{2}\mathbf{K}\right)\mathbf{c}^n \tag{2.16}$$

6. **迭代求解**：在每个时间步内迭代求解该系统

优势:

- **几何灵活性**：可处理不规则的边界条件
- **自适应加密**：在势场剧烈变化区域自动加密网格（如量子点边界）
- **精度可控**：通过提高单元阶数（p-refinement）实现指数收敛
- **并行效率**：现代 FEM 库（如 FEniCS, Deal.II）支持大规模并行计算

劣势:

- 高维情形计算储存量大，并且对于连续谱处理很困难
- 同时也十分依赖边界条件和初始条件

2.2.4 谱方法[14]

核心思想：将波函数展开为一组已知正交基函数的线性组合，并且通过截断展开项将微分方程展开为代数方程求解：

$$\psi(x) = \sum_n c_n \phi_n(x) \tag{2.17}$$

代入薛定谔方程后，通过 Galerkin 投影得到系数方程。

实施步骤　一般采取如下正交基函数:

- 傅里叶基 e^{ikx}
- 正弦函数 $\sin(n\pi x/L)$
- Hermite 多项式 $H_n(x) = e^{-x^2/2}$

再将波函数 $\psi(x)$ 展开成正交基函数的线性组合：$\psi(x) \approx \sum_{n=1}^{N} c_n \phi_n(x)$, 其中，N 为截断精度。将薛定谔方程 $\hat{H}\psi = E\psi$ 投影到每个基函数 ϕ_m 上，并且转化为矩阵本征值问题：

$$\sum_{n=1}^{N} c_n \langle \phi_m | \hat{H} | \phi_n \rangle = E \sum_{n=1}^{N} c_n \langle \phi_m | \phi_n \rangle \tag{2.18}$$

利用基函数的正交性 $\langle \phi_m | \phi_n \rangle = \delta_{mn}$ 得到广义本征问题:

$$\mathbf{Hc} = E\mathbf{S}c \tag{2.19}$$

- 哈密尔顿矩阵:$H_{mn} = \langle \phi_m | \hat{H} | \phi_n \rangle$
- 重叠矩阵:$S_{mn} = \langle \phi_m | \phi_n \rangle$

之后我们就将求解薛定谔方程转化为直接求解矩阵方程。

优势：

- 若解光滑，收敛速度快（指数级）
- 转化为矩阵方程组求解，相对高效

劣势：

- 不容易处理非光滑解的情形
- 非均匀或复杂几何问题可能需要其他基函数

2.2.5 分析与讨论

综合所有传统的数值方法，它们通过将方程转化为离散问题，进而通过离散化方法进行求解。此外，这些方法在处理某些特定问题时具有明显优势。例如，有限元法（FEM）在几何适应性方面表现突出，特别适用于解决复杂边界问题；谱方

法适合周期性系统；变分法在求解基态问题时效率较高；有限差分法（FDM）则适用于均匀网格和规则区域。然而，这些方法都有一个共同的问题，那就是在高维情况下，它们会面临计算量急剧增加的挑战。尤其是对于 FDM，网格点数随着维度的增加呈指数增长，而谱方法的基函数数量也会随着维度的上升急剧增加。因此，当求解高维度的薛定谔方程时，传统方法面临着巨大的挑战，这就需要引入新的方法来应对。为此，物理信息神经网络（PINN）方法应运而生，用于求解薛定谔方程。在介绍 PINN 方法之前，我们首先引入 WKB 解的概念

2.3 WKB 近似 (WKB approximation)[22]

为了获得解 u^ϵ 的具有震荡结构的解，根据 Carlos 的在 2008 年的文章[10]，我们假设解 u^ϵ 具有以下形式，即 WKB 近似 (Wentzel-Kramers-Brillouin approximation):

$$u^\epsilon(t,x) \overset{\epsilon\to 0}{\sim} a^\epsilon(t,x)e^{iS(t,x)/\epsilon}$$

其中 S 是实值相位函数 (real-valued phase function)，a^ϵ 是复值振幅 (complex-valued amplitude)，满足：

$$u^\epsilon(t,x) \overset{\epsilon\to 0}{\sim} a+\epsilon a_1+\epsilon^2 a_2+\cdots$$

也就是可以在 $\epsilon\to 0$ 的情况下，可以对 $u^\epsilon(t,x)$ 进行展开。之后，我们将 WKB 近似代回薛定谔方程，并对每阶 ϵ 进行求解：

$$i\epsilon\partial_t(a^\epsilon(t,x)e^{iS(t,x)/\epsilon})=-\frac{\epsilon^2}{2}\Delta(a^\epsilon(t,x)e^{iS(t,x)/\epsilon})+V(x)a^\epsilon(t,x)e^{iS(t,x)/\epsilon}, u^\epsilon(0,x)=u_{in}^\epsilon(x)$$

若我们考虑一阶 ϵ：

$$\begin{aligned} i\epsilon\partial_t u^\epsilon &= -\frac{\epsilon^2}{2}\Delta u^\epsilon+V(x)u^\epsilon, u^\epsilon(0,x)=u_{in}^\epsilon(x),\\ u^\epsilon &= (a+\epsilon a_1)e^{iS/\epsilon},\\ -\epsilon a_1 S_t e^{iS/\epsilon}+i\epsilon a_t e^{iS/\epsilon} &= V(x)a_1e^{iS/\epsilon}-(a_t\frac{iS_t}{\epsilon}e^{iS/\epsilon}+a_t\frac{iS_t}{\epsilon}e^{iS/\epsilon}+a\frac{i}{\epsilon}S_{tt}e^{iS/\epsilon}-a_1\frac{(S_t)^2}{\epsilon}e^{iS/\epsilon}) \end{aligned}$$

最后，我们取所有 0 阶 ϵ 的项，得到相位函数 S 满足的 Hamilton-Jacobi 方程，即：

$$\begin{aligned} LHS &= i\epsilon\partial_t[(a+\epsilon a_1+\cdots)e^{iS/\epsilon}],\\ &= i\epsilon\cdot a\frac{S}{\epsilon}S_t\cdot e^{iS/\epsilon}\\ RHS &= \frac{1}{2}a\cdot e^{iS/\epsilon}|S_t|^2+V(x)\cdot ae^{iS/\epsilon} \end{aligned}$$

取所有 0 阶 ε 项可得:

$$\partial_t S + \frac{1}{2}|\nabla S|^2 + V(x) = 0, S(0,x) = S_{in}(0,x)$$

在函数 $V(x)$ 充分光滑的前提下，即 $V(x) \in C^2(\mathbb{R}^d)$，该方程可以用特征线法求解，其特征流 (characteristic flow) 由以下哈密顿系统给出:

$$\begin{aligned}
&\dot{x}(t,y) = \xi(t,y), x(0,y) = y,\\
&\dot{\xi}(t,y) = -\nabla_x V(x(t,y)), \xi(0,y) = \nabla S_{in}(y).
\end{aligned}$$

根据 Cauchy-Lipschitz 定理，该常微分方程组在可以在局部时间内求解:

$$S(t,x) = S(0,x) + \int_0^t \frac{1}{2}|\nabla S(\tau, y(t,x))|^2 - V(\tau, y(t,x))d\tau \tag{2.20}$$

其中 $y(\tau,x)$ 表示特征流 (characteristic flow) $X_t : y \mapsto x(t,y)$ 的逆映射。这产生一个光滑的相位函数 $S \in C^\infty([-T,T]\times\mathbb{R}^d)$，其中 $T > 0$ 但可能其值很小。

然后，我们取 1 阶 ε 项，可得:

$$\partial_t a + \nabla S \cdot \nabla a + \frac{a}{2}\Delta S = 0, \quad a(0,x) = a_{in}(x)$$

进一步化简，取 $\rho := |a|^2$ ，那么 ρ 满足:

$$\partial_t \rho + \operatorname{div}(\rho \nabla S) = 0$$

这个方程类似一个守恒定律。同样，该方程可以被特征线法求解，得到:

$$a(t,x) = \frac{a_{in}(y(t,x))}{\sqrt{J_t(y(t,x))}}, \quad |t| \le T$$

其中 $J_t(y) := \det \nabla_y x(t,u)$ 是哈密顿流的雅可比行列式。

递推关系 如果进一步求解每一阶的 ε 项，我们就可以获取各阶振幅 a_n 的形式，满足下列递推关系式:

$$\partial_t a_n + \nabla S \cdot \nabla a_n + \frac{a_n}{2}\Delta S = \Delta a_{n-1}$$

这些方程仍然可以依次通过特征线法求解。因此, 在至少局部时间内, 即焦散 (caustics) 发生前, 各阶振幅都有解。从而，这也最终给出一个 WKB 形式的近似解:

$$u^\varepsilon_{app}(t,x) = e^{iS(t,x)/\varepsilon}(a(t,x) + \varepsilon a_1(t,x) + \varepsilon^2 a_2(t,x) + \cdots)$$

定理 2.1 在给定薛定谔方程的初始数据的情况下，并且假设初始数据具有 WKB 形式：

$$u_{in}^{\varepsilon}(x) = a_{in}e^{iS_{in}(x)/\varepsilon}$$

其中 $S_{in} \in C^{\infty}(\mathbb{R}^d)$，且 $a_{in} \in \mathbf{S}(\mathbb{R}^d)$。则在焦散之前，对于任意闭时间区间 $I \subset T$，存在常数 $C > 0$（独立于 $\varepsilon \in (0, 1]$），使得：

$$\sup_{t\in I} ||u^{\varepsilon}(t) - u_{app}^{\varepsilon}(t)||_{L^2\cap L^{\infty}} \leq C\varepsilon^N$$

焦散以及 WKB 解的优势和缺陷 在展开 WKB 解的优势与缺陷之前，我们先介绍一下焦散面 (caustics)。量子物理中，焦散面对应经典粒子轨迹（或光线）在相空间中的拉格朗日子流形投影到实空间时产生的折叠奇点，而它的数学的定义在这里如下：

定义 2.2 焦散集 (caustics set)：特征线在有限时间 $|T| < \infty$ 处会相交，此时流映射 $X_t : \mathbb{R}^d \to \mathbb{R}^d$ 不再是单射 (参见 2.1 式由 $y(\tau, x)$ 表示的特征流 X_t)。使得 X_t 失去微分同胚性质的点的集合被称为 **焦散集**。

WKB 近似解的优势在于，它避免了直接求解复杂的薛定谔方程，并提供了直观的解析解形式，显式关联相位函数 $S(x,t)$ 和振幅 $a(x,t)$，直接反映了粒子轨迹，即经典动量 ∇S 与量子振幅的调制关系；并且在 ε 很小的时候，可以近似估计薛定谔方程的解 (参考定理 2.1)。

但是，WKB 解也有它的缺陷，就是焦散面对于解的影响：从物理学的角度来说，焦散面标记了经典粒子轨迹与量子波动性显著耦合的区域，表示了半经典极限的临界点，从数学的角度来说，函数 $S(x,t)$ 的梯度 ∇S 在焦散面处发散，这说明 WKB 解出现了新的 ε 震荡尺度，同时也会导致 WKB 解失去精准性。

正因为 WKB 解并不完美，人们一直在寻求更为精确的方程求解方法，这也促成了物理信息神经网络（PINN）方法的提出：

2.4 物理信息神经网络 (Physics-Informed Neural Networks, PINN)

物理信息神经网络（Physics-Informed Neural Networks, PINN）是一种将物理规律嵌入神经网络架构的机器学习方法，通过将偏微分方程（PDE）的数学约束直接引入损失函数，使网络在训练中同时拟合数据和满足物理规律。这种神经网络方法可以有效的克服传统数值方法在高维、复杂边界或数据稀缺场景中的局限性。

2.4.1 起源

物理信息神经网络 (之后就用 PINN 代指) 的概念最早由 Raissi 等人在 2019 年提出[16]。他们发现传统深度学习方法（如 CNN/RNN 等）在数据稀缺时缺乏鲁棒性，难以保证收敛性，于是他们将物理定律、经验规则或领域知识作为正则化手段，从而缩小解空间并提升模型计算效率。

2.4.2 模型构建

[20]首先，我们考虑偏微分方程 (以后简称 pde) 的一般形式:

$$u_t + \mathcal{N}[u] = 0, \quad x \in \Omega,\ t \in [0, T], \tag{2.21}$$

其中，$u(t, x)$ 表示隐式解 (latent solution), $\mathcal{N}[\cdot]$ 是一个非线性微分算子 (nonlinear differential operator), 且 $\Omega \subseteq \mathbb{R}^D$. 之后，我们会对 pde 的模型进行分类，分别是时间连续模型 (continuous time model) 和离散时间模型 (discrete time model)：

连续时间模型 (continuos time model) 我们定义方程 (2.2) 左端为 $f(t, x)$，即:

$$f := u_t + \mathcal{N}[u] = 0, \quad x \in \Omega,\ t \in [0, T] \tag{2.22}$$

并且通过深度神经网络去估计 $u(t, x)$。这个假设以及函数 (2.3) 共同构成了物理信息神经网络，该网络可以通过应用使用自动微分 (automatic differentiation) 对该复合函数使用链式法则微分得到[17]，并且与表示 $u(t, x)$ 的网络具有相同的参数，尽管由于微分算子 N 的作用而具有不同的激活函数。神经网络 $u(t, x)$ 和 $f(t, x)$ 之间共享的参数可以通过最小化它们的均方误差 (Mean Squared Loss,MSE) 损失来训练。

$$\text{MSE} = \text{MSE}_u + \text{MSE}_f, \tag{2.23}$$

其中:

$$\text{MSE}_u = \frac{1}{N_u}\sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2, \text{MSE}_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2 \tag{2.24}$$

$\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ 表示 $u(t, x)$ 在初始条件和边界数据，而 $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ 表示了 $f(t, x)$ 的配置点 (collocation points)，损失函数 MSE_u 对应初始条件和边界数据，损失函数 MSE_f 则通过了有限的配置点刻画了方程 2.3 的结构。

以带周期性边界条件的非线性薛定谔方程为例：

$$ih_t + 0.5h_{xx} + |h|^2 h = 0, \quad x \in [-5, 5], \quad t \in [0, \pi/2] \tag{2.25}$$

$$h(0, x) = 2\,\mathrm{sech}(x) \tag{2.26}$$

$$h(t, -5) = h(t, 5) \tag{2.27}$$

$$h_x(t, -5) = h_x(t, 5) \tag{2.28}$$

其中 $h(t, x)$ 为复数值解。定义微分算子：

$$f(t, x) := ih_t + 0.5h_{xx} + |h|^2 h \tag{2.29}$$

我们根据 $h(t, x)$ 构建**复值神经网络**。事实上，若 $h(t, x)$ 是复值函数，令 u 表示 $h(t, x)$ 的实值部分，v 表示 $h(t, x)$ 的复值部分，那么，我们可以基于

$$h(t, x) = \begin{bmatrix} u(t, x) & v(t, x) \end{bmatrix} \tag{2.30}$$

构建多输出物理信息神经网络 $f(t, x)$。通过最小化复合 $f(t, x)$ 和 $h(t, x)$ 的损失函数学习它们的共享参数：

$$\mathrm{MSE} = \mathrm{MSE}_0 + \mathrm{MSE}_b + \mathrm{MSE}_f \tag{2.31}$$

其中，各损失项定义如下：

$$\mathrm{MSE}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |h(0, x_i^b) - h_0^i|^2 \tag{2.32}$$

$$\mathrm{MSE}_b = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(|h^i(t_i^b, -5) - h^i(t_i^b, 5)|^2 + |h_x^i(t_i^b, -5) - h_x^i(t_i^b, 5)|^2 \right) \tag{2.33}$$

$$\mathrm{MSE}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2 \tag{2.34}$$

其中：

- $\{x_i^b, h_0^i\}_{i=1}^{N_0}$：初始条件数据点
- $\{t_i^b\}_{i=1}^{N_b}$：边界配置点
- $\{t_f^i, x_f^i\}_{i=1}^{N_f}$：PDE 的配置点

各项损失函数的物理意义如下：

- MSE_0：用于拟合初始条件
- MSE_b：用于刻画周期边界条件
- MSE_f：用于惩罚不满足 PDE 的情况

离散时间模型 (discrete time model) 由于离散时间模型并不是求解薛定谔方程的主要内容，故此只做简单陈述：我们首先将具有 q 级的 Runge-Kutta 方法的一般形式[24]应用于方程 (2.20)，得到：

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}; \lambda], \quad i = 1, \ldots, q, \tag{2.35}$$

$$u^{n+1} = u^n - \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u^{n+c_j}; \lambda]. \tag{2.36}$$

此处，$u^{n+c_j}(x) = u(t^n + c_j \Delta t, x)$ 表示了该上述系统在时间 $t^n + c_j \Delta t$ 处的隐藏状态，其中 $j = 1, \ldots, q$。这一通用形式涵盖了隐式和显式时间步进方案，具体取决于参数 $\{a_{ij}, b_j, c_j\}$ 的选择。方程 (2.31)，(2.32) 可以等价地表示为：

$$u^n = u_i^n, \quad i = 1, \ldots, q, \tag{2.37}$$

$$u^{n+1} = u_i^{n+1}, \quad i = 1, \ldots, q, \tag{2.38}$$

其中：

$$u_i^n := u^{n+c_i} + \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}; \lambda], \quad i = 1, \ldots, q, \tag{2.39}$$

$$u_i^{n+1} := u^{n+c_i} + \Delta t \sum_{j=1}^{q} (a_{ij} - b_j) \mathcal{N}[u^{n+c_j}; \lambda], \quad i = 1, \ldots, q. \tag{2.40}$$

然后，我们对以下变量使用多输出神经网络：

$$\left[u^{n+c_1}(x), \ldots, u^{n+c_q}(x)\right]. \tag{2.41}$$

这一先验假设与方程 (2.35),(2.36) 共同构成了两个物理信息神经网络：

$$\left[u_1^n(x), \ldots, u_q^n(x), u_{q+1}^n(x)\right], \tag{2.42}$$

和

$$\left[u_1^{n+1}(x), \ldots, u_q^{n+1}(x), u_{q+1}^{n+1}(x)\right]. \tag{2.43}$$

给定系统在两个不同时刻 $\{x^n, u^n\}$ 和 $\{x^{n+1}, u^{n+1}\}$ 的噪声测量数据，分别对应时间 t^n 和 t^{n+1}，可以通过最小化平方误差和来训练神经网络 (2.37),(2.38),(2.39) 的共享参数以及微分算子的参数 λ：

$$SSE = SSE_n + SSE_{n+1}, \tag{2.44}$$

其中：

$$SSE_n := \sum_{j=1}^{q} \sum_{i=1}^{N_n} |u_j^n(x^{n,i}) - u^{n,i}|^2, \tag{2.45}$$

$$SSE_{n+1} := \sum_{j=1}^{q} \sum_{i=1}^{N_{n+1}} |u_j^{n+1}(x^{n+1,i}) - u^{n+1,i}|^2. \tag{2.46}$$

并且，$x^n = \{x^{n,i}\}_{i=1}^{N_n}$，$u^n = \{u^{n,i}\}_{i=1}^{N_n}$，$x^{n+1} = \{x^{n+1,i}\}_{i=1}^{N_{n+1}}$，且 $u^{n+1} = \{u^{n+1,i}\}_{i=1}^{N_{n+1}}$。

2.4.3 实验结果与比较

同时，也对 PINN 方法的精度进行评估，我们采用传统谱方法对方程 (2.7) 进行高分辨率数值模拟，在同样的初始条件和边界条件的情况下，即：

- **初始条件**: $h(0, x) = 2\,\mathrm{sech}(x)$
- **边界条件**: 周期性边界 $h(t, -5) = h(t, 5)$ 和 $h_x(t, -5) = h_x(t, 5)$

并且采用如下的传统数值方案:

- 通过 Chebfun 包[23]实现 256 模谱傅里叶离散化
- 四阶显式 Runge-Kutta 时间积分 ($\Delta t = \pi/2 \times 10^{-6}$)

与 PINN 进行对比，采点为：

- $N_0 = 50$(取 $t = 0$ 时刻的初始条件上的点);
- $N_b = 50$(取边界条件上的点)；
- $N_f = 20000$(取此配置点在解的领域来训练方程)

我们把 $h(t, x)$ 用 $u(t, x), v(t, x)$ 联合表示出来

$$h(t, x) = [u(t, x)\ v(t, x)] \approx \mathrm{NN}(t, x; \theta)$$

采用如下神经网络的配置:

- 5 层全连接网络 (每层 100 神经元)
- 激活函数: tanh

对方程的求解进行比较

2.4.4 问题与讨论

方程 (2.27) 中的 MSE_f 项作为正则化机制，惩罚不满足控制方程 (2.25) 的解。这使得物理信息神经网络具有：

优势　由于方程 (2.9) 中的 MSE_f 项作为一种正则化机制，惩罚不满足方程 (2.7) 的解, 所以用 PINN 去求解薛定谔方程会有以下优势：

- 小样本训练能力 (适应高成本实验数据场景)
- 实测相对 L_2 误差: 1.97×10^{-3}

劣势

- **当前局限**:
  - 高维问题需要指数增长的配置点 N_f
  - 时空全局约束计算成本高
- **改进方案**:
  - 可以尝试采用稀疏网格或者拟蒙特卡洛采样
  - 基于 Runge-Kutta 时间离散的采用新型网络架构

2.5 使用 PINN 去获得 WKB 解

首先，在求解薛定谔方程的时候，我们可以使用 WKB 解去近似薛定谔方程的近似解：

$$u^{\varepsilon}(t,x) \overset{\varepsilon \to 0}{\sim} a^{\varepsilon}(t,x) e^{iS(t,x)/\varepsilon} \tag{2.47}$$

其中，

$$S(t,x) = S(0,x) + \int_0^t \frac{1}{2} |\nabla S(\tau, y(t,x))|^2 - V(\tau, y(t,x)) d\tau \tag{2.48}$$

以及

$$\partial_t a + \nabla S \cdot \nabla a + \frac{a}{2} \Delta S = 0, \quad a(0,x) = a_{in}(x) \tag{2.49}$$

$$\nabla_t a_n + \nabla S \cdot \nabla a_n + \frac{a_n}{2} \Delta S = \Delta a_{n-1} \tag{2.50}$$

但是我们发现 $S(t,x), a_n, n \in N$ 都仅仅是形式上的积分求解，实际计算出来需要很大的力气，并且在高维度的情形下，对于 $S(t,x), a_n, n \in N$ 复杂积分的数值计算十分的低效，所以，我将使用 PINN 去获得 WKB 解。

2.6 基于物理信息神经网络的 WKB 近似求解方法

我们采用物理信息神经网络（Physics-Informed Neural Networks, PINNs）结合 WKB 渐近展开，求解含时薛定谔方程：

$$i\epsilon \frac{\partial \psi}{\partial t} = -\frac{\epsilon^2}{2} \nabla^2 \psi + V(x)\psi \tag{2.51}$$

其中，为了求解简便，同时我们在实验中暂时使用不到更低阶的 ϵ 项，我将波函数按 WKB 展开为：

$$\psi(x,t) = e^{iS(x,t)/\epsilon} \left[a(x,t) + \epsilon a_1(x,t) + \mathcal{O}(\epsilon^2) \right] \tag{2.52}$$

我们不会考虑 ε^2 以上的项，在后续的数值计算中，我们会采用 $\varepsilon = 0.01$ 来求出 WKB 解。

2.6.1 网络架构设计

- **初值条件**：在求解偏微分方程的时候，我们需要初始条件，根据 WKB 近似的假设,:

$$u_{in}^{\varepsilon}(x) = a_{in}(x)e^{iS_{in}(x)/\varepsilon} \tag{2.53}$$

其中 $u_{in}^{\varepsilon}(x)$ 是薛定谔方程的初始条件，所以为了方便起见，在后续的计算中，我将直接给出 $S_{in}(x), a_{in}(x), a_{1in}(x)$ 的初值 $V(x)$ 的初值条件也已经给出:

$$a_{in}(x) = e^{-x^2} \tag{2.54}$$

$$V(x) = \frac{1}{2}x^2 \tag{2.55}$$

$$S_{in}(x) = sin(x) \tag{2.56}$$

$$a_{1in}(x) = sin(x) \tag{2.57}$$

- **分阶段建模**：我将构建三个独立的 PINN 来分别求解相位 $S(x,t)$、首项振幅 $a(x,t)$ 和一阶修正项 $a_1(x,t)$
- **网络结构**：每个模型采用 4 层全连接网络，隐藏层维度为 64，激活函数为双曲正切函数:

$$\mathcal{NN}(x,t) = W_4 \circ \tanh \circ W_3 \circ \tanh \circ W_2 \circ \tanh \circ W_1(x,t) \tag{2.58}$$

其中 W_i 表示线性变换层

2.6.2 损失函数构建

在每个 PINN 中，对每个物理量实施分步训练策略:

1. **相位 S 的求解**:

$$\mathcal{L}_S = \underbrace{\frac{1}{N_f}\sum_{i=1}^{N_f}\left|\frac{\partial S(x_i,t)}{\partial t} + \frac{1}{2}\left(\frac{\partial S(x_i,t)}{\partial x}\right)^2 + V(x_i)\right|^2}_{\text{Hamilton-Jacobi 方程残差}} + \underbrace{\frac{1}{N_0}\sum_{i=1}^{N_0}|S(x_i,0) - S_{in}(x_i)|^2}_{\text{初始条件}} \tag{2.59}$$

2. **振幅 a 的求解**（冻结 S 网络参数）:

$$\mathcal{L}_a = \underbrace{\frac{1}{N_f}\sum\left|\frac{\partial a}{\partial t} + \frac{\partial S}{\partial x}\frac{\partial a}{\partial x} + \frac{a}{2}\frac{\partial^2 S}{\partial x^2}\right|^2}_{\text{a 的递推方程}} + \underbrace{\frac{1}{N_0}\sum|a(x,0) - a_{in}(x)|^2}_{\text{初始条件}} \tag{2.60}$$

3. **修正项 a_1 的求解**（冻结 S, a 参数）：

$$\mathcal{L}_{a_1} = \underbrace{\frac{1}{N_f}\sum_{i=1}^{N_f}\left|\frac{\partial a_1(x_i,t)}{\partial t} + \frac{\partial S(x_i,t)}{\partial x}\frac{\partial a_1(x_i,t)}{\partial x} + \frac{a_1(x_i,t)}{2}\frac{\partial^2 S(x_i,t)}{\partial x^2} - \frac{\partial^2 a(x_i,t)}{\partial x^2}\right|^2}_{a_1\text{的递推方程}} \tag{2.61}$$

$$+ \underbrace{\frac{1}{N_0}\sum_{i=1}^{N_0}|a_{1in}(x_i,0) - a_{1in}(x_i)|^2}_{a_1\text{的初始条件}} \tag{2.62}$$

2.6.3 训练策略

- **神经网络结构：** 采用 4 层全连接的前馈神经网络
- **输入层：** 1 层
- **隐藏层：** 3 层，每层 64 个神经元
- **激活函数：** tanhx
- **输出层：** 1 层

Algorithm 1 分阶段 PINN 训练流程

1: **训练主势函数 $S(x,t)$**
2: 初始化网络参数 θ_S
3: **while** 未达到收敛条件 **do**
4: 在域内采样配点 $\{(x_f^i, t_f^i)\}_{i=1}^{N_f}$
5: 在初始时刻采样点 $\{(x_0^i, 0)\}_{i=1}^{N_0}$
6: 计算损失：$\mathcal{L}_S = \frac{1}{N_f}\sum_{i=1}^{N_f}\left|\frac{\partial S(x_i,t)}{\partial t} + \frac{1}{2}\left(\frac{\partial S(x_i,t)}{\partial x}\right)^2 + V(x_i)\right|^2 + \frac{1}{N_0}\sum_{i=1}^{N_0}|S(x_i,0) - S_{in}(x_i)|^2$
7: 更新参数：$\theta_S \leftarrow (\theta_S - \eta\nabla_{\theta_S}\mathcal{L}_S)$
8: **end while**
9: 冻结参数 θ_S

其余对于 a 和 a_1 的计算也是同理，在此省略。

2.6.4 数值实现细节

- 采用自动微分技术计算偏导数 $\partial/\partial x, \partial/\partial t$
- 使用 Adam 优化器，学习率 10^{-3}，每阶段训练 5000 轮

- 配点采样区域：空间域 $x \in [-1, 1]$，时间域 $t \in [0, 1]$
- 最终解重构：

$$\psi_{\text{WKB}}(x, t) = (a + \epsilon a_1)e^{iS/\epsilon} \tag{2.63}$$

配点采样： 在计算域内随机采样训练点：

- **S 模型**：1000 个配点（$N_f = 1000$），空间范围 $x \in [-1, 1]$，时间范围 $t \in [0, 1]$
- **a 模型**：2000 个配点（$N_{f_a} = 2000$），采样范围与 S 模型相同
- a_1 **模型**：3000 个配点（$N_{f_{a_1}} = 3000$），采样范围与 S 模型相同

初始条件： 初始时刻（$t = 0$）的采样点设置：

- **S 模型**：200 个初始点（$N_0 = 200$），空间范围 $x \in [-1, 1]$
- **a 模型**：500 个初始点（$N_{0a} = 500$），空间范围 $x \in [-1, 1]$
- a_1 **模型**：800 个初始点（$N_{0a_1} = 800$），空间范围 $x \in [-1, 1]$

参数冻结： 为了保留在之前模型中求出的参数，故我在训练高阶模型时进行参数冻结：

- 训练 **a 模型**时，冻结已训练好的 **S 模型**参数
- 训练 a_1 **模型**时，同时冻结 **S 模型**和 **a 模型**参数

可视化： 通过 Matplotlib 实现可视化：

- 固定 t 时，解 $u(x, t)$ 随空间 x 的变化曲线

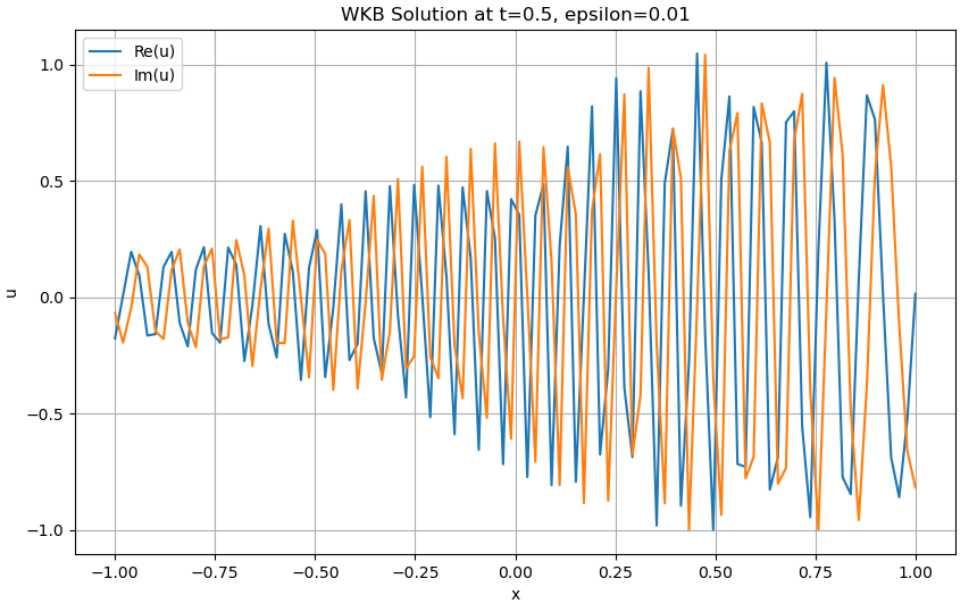


图 2.1　WKB Solution at $t = 0.5, \varepsilon = 0.01$

- 固定 x 时, 解 $u(x,t)$ 随时间 t 的演化曲线

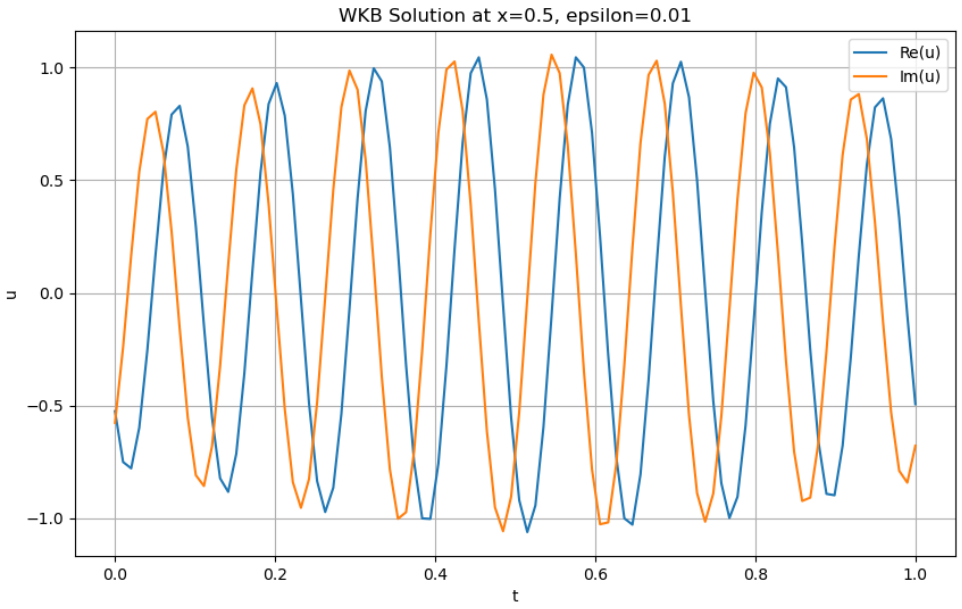


图 2.2　WKB Solution at $x = 0.5, \varepsilon = 0.01$

- WKB 近似解的 3D 曲面图 ($Re(u)$)
- WKB 近似解的 3D 曲面图 ($Im(u)$)

2.6.5　与谱方法对比

我们将采用谱方法去获得薛定谔方程的解，和我们所求的 WKB 解进行对照，现在简单阐述谱方法的基本原理以及此处谱方法[15]的优势:

基本原理

- 1. **基函数展开:** 把所需要求解的 $u^{\epsilon}(t,x)$ 用基函数 e^{ik} 展开
- 2. **微分运算转化为代数运算:** 这里利用 Fourier 变换，将半经典薛定谔方程中的微分算子转换为对应 Fourier 变换后空间中的乘法运算，从而进行数值计算

数值实现:　首先，保证和 PINN 方法求解 WKB 解的初值条件一样, 根据 (2.49) − (2.52) 式

- $V(x) = \frac{1}{2}x^2$

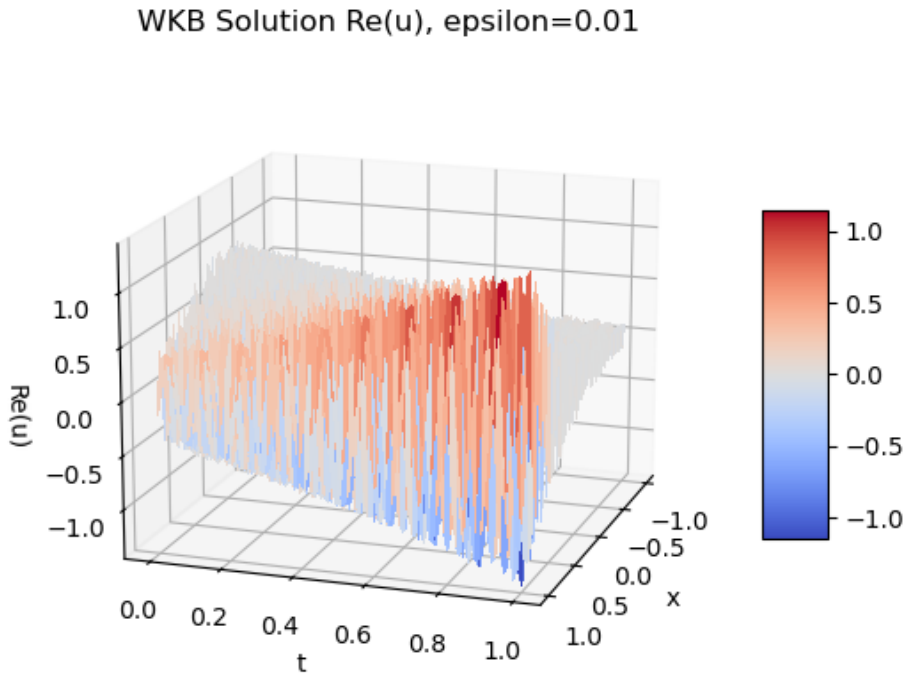


图 2.3　WKB Solution $Re(u), \varepsilon = 0.01$

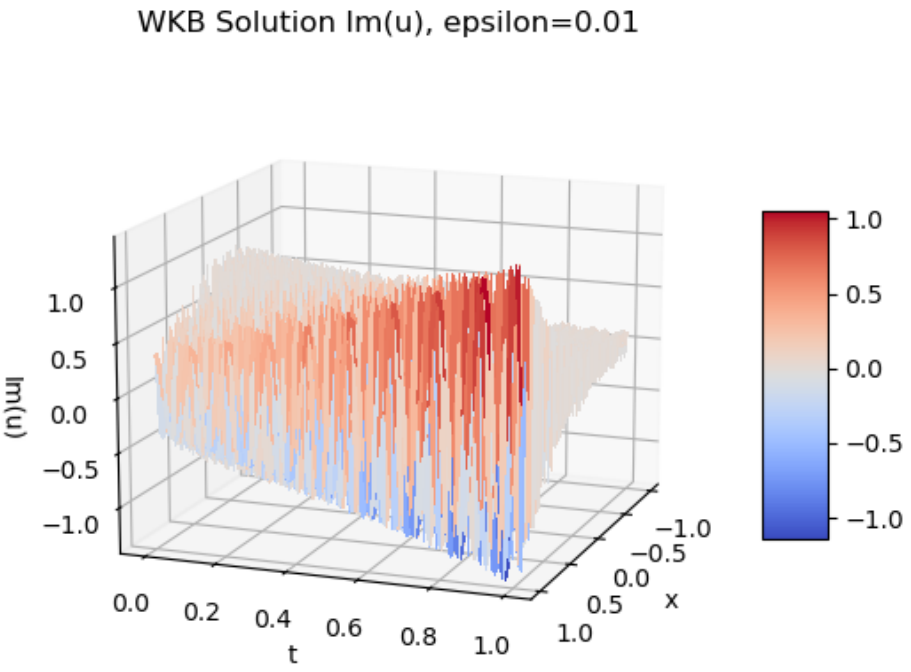


图 2.4　WKB Solution $Im(u), \varepsilon = 0.01$

- $t = 0$ 时

$$u^{\varepsilon}(0,x) = u_{in}^{\varepsilon}(x) = (a_{in}(x) + a_{1\,in}(x)\varepsilon) \cdot e^{iS_{in}(x)/\varepsilon} \tag{2.64}$$

$$= (e^{-x^2} + \varepsilon \cdot sin(x))e^{isin(x)/\varepsilon} \tag{2.65}$$

- 基函数:e^{ikx}
- 时间步长:$\Delta t = 0.01$, 总时间:$T = 2.0$
- 空间域:$x \in [-20, 20]$, 均匀划分 $N_x = 2^{12}$ 个点

可视化 通过 Matlpotlib 实现可视化:

- 固定 t 时, 解 $u(x,t)$ 随空间 x 的变化曲线

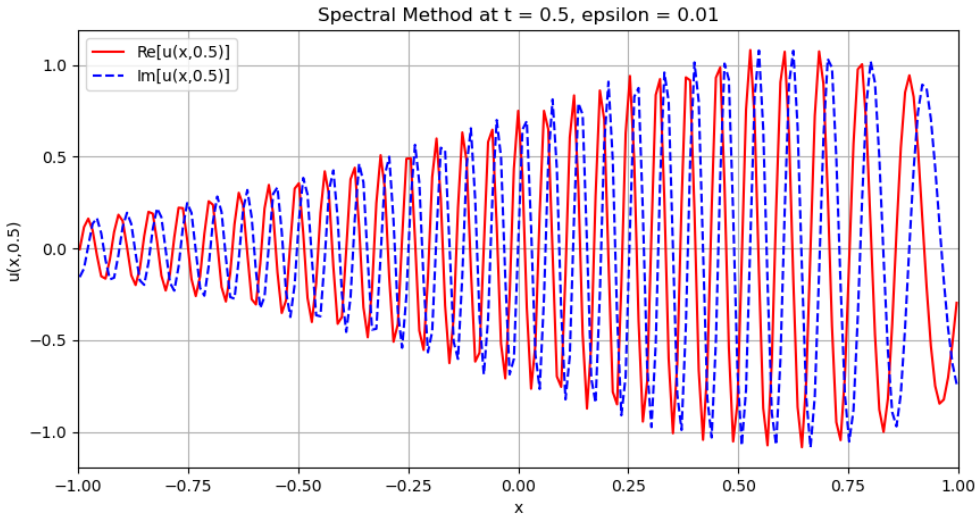


图 2.5 Spectral Method at $t = 0.5, \varepsilon = 0.01$

- 固定 x 时, 解 $u(x,t)$ 随时间 t 的演化曲线
- 谱方法所求解的 3D 曲面图 $Re(u)$

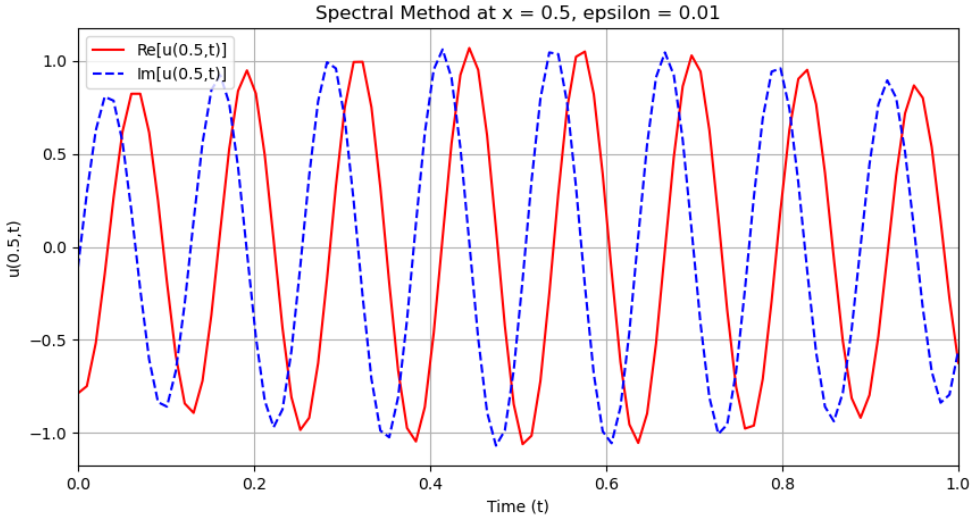


图 2.6　Spectral Method at $x = 0.5, \varepsilon = 0.01$

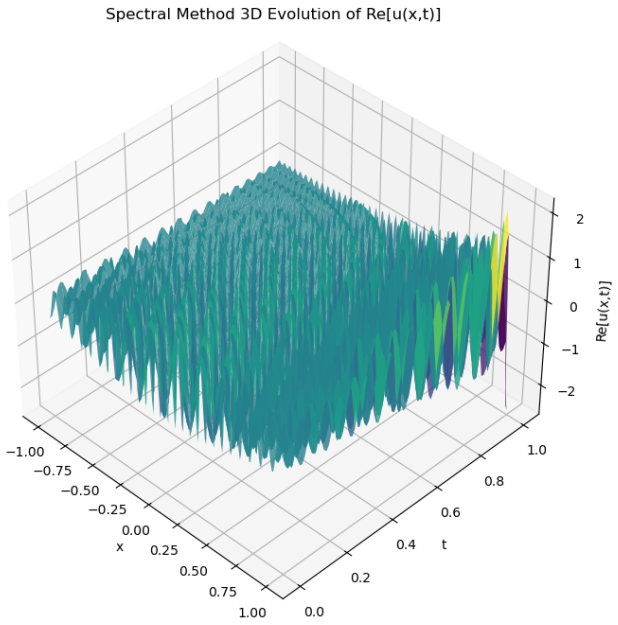


图 2.7　Spectral Method $Re(u), \varepsilon = 0.01$

- WKB 近似解的 3D 曲面图 $Im(u)$

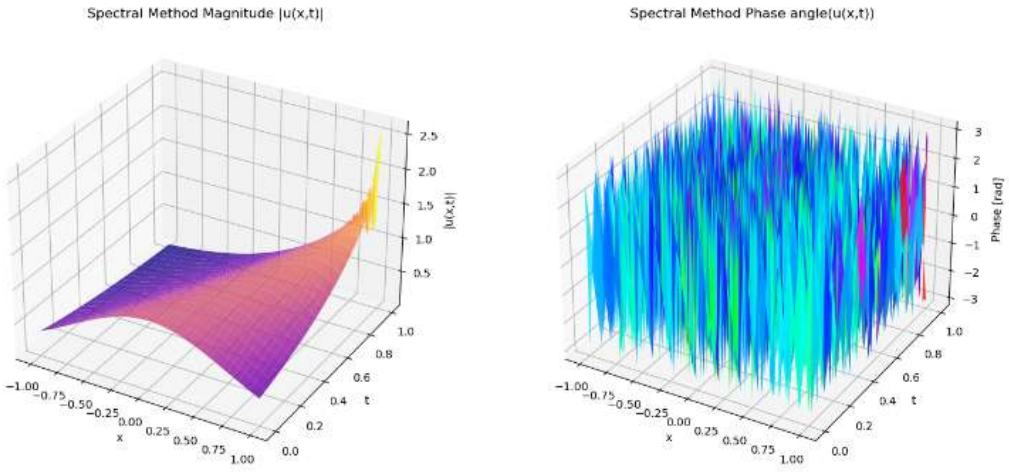


图 2.8　Spectral Method $Im(u), \varepsilon = 0.01$

谱方法所求解和 WKB 近似解的对比图　我们固定 x=0.5 可以得到二者的绝对误差的对比图像:

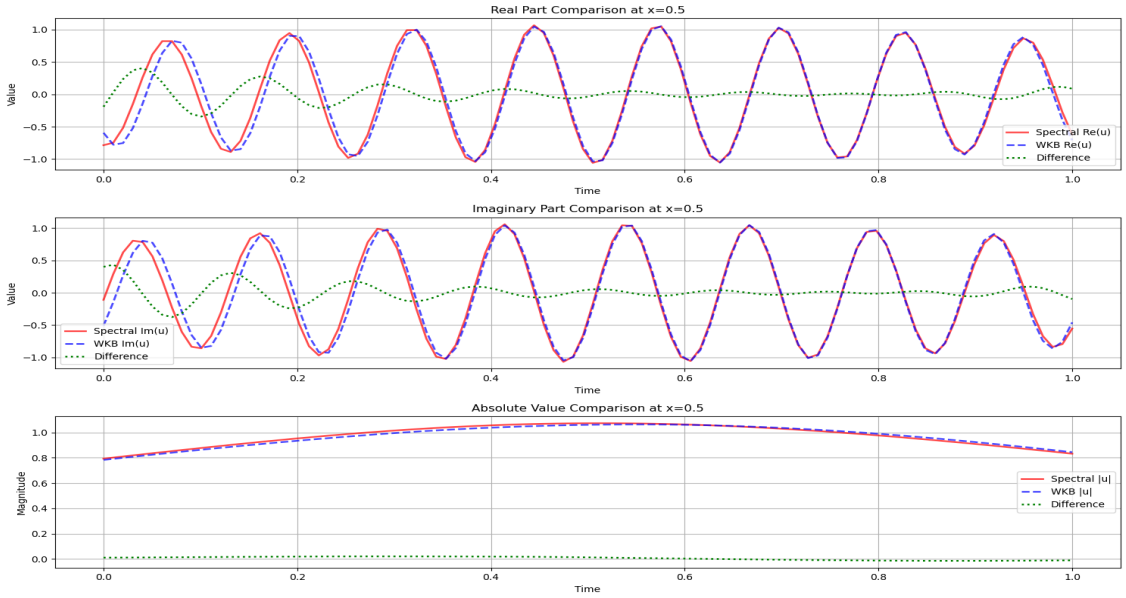


图 2.9　Comparison $Re(u), \varepsilon = 0.01$

并且计算二者的 L^2 误差:

$$L^2\text{范数}：8.0569 \times 10^{-2} - 2.5906 \times 10^{-1} i \tag{2.66}$$

同样，在固定 t=0.5 时，我们可以得到二者的绝对误差的对比图像:

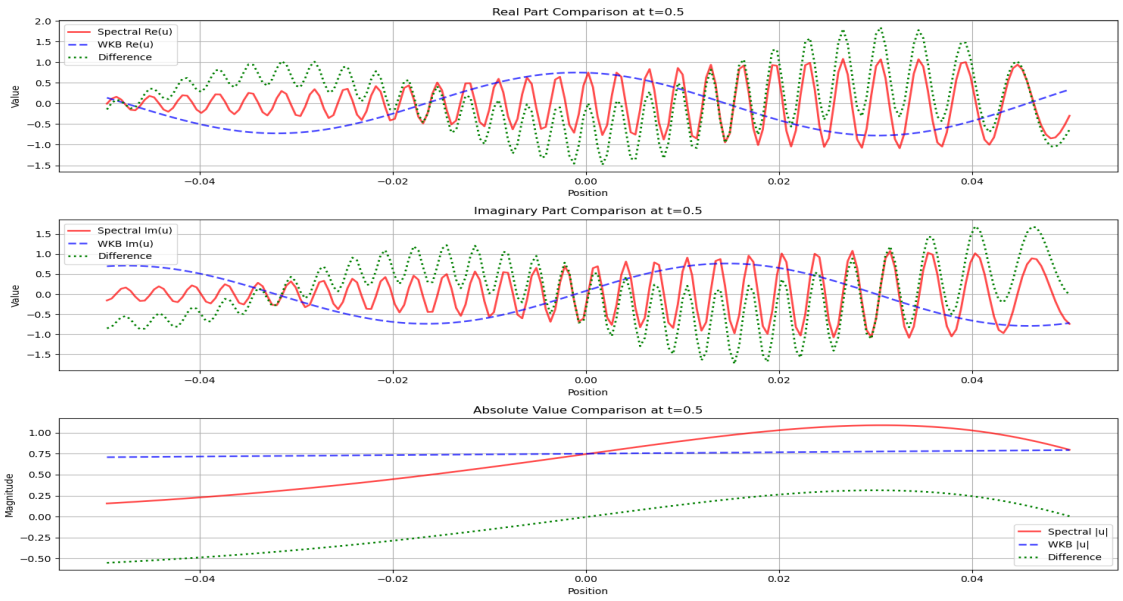


图 2.10　Comparison $Im(u), \varepsilon = 0.01$

并且计算二者的 L^2 误差:

$$L^2\text{范数}：2.7609 \times 10^{-1} + 2.1406 \times 10^{-1} i \tag{2.67}$$

2.6.5.1　分析与总结

[21] [22]根据本文及相关背景知识，我们知道半经典薛定谔方程的解呈现高频振荡特性。我们所选择的分别固定在 $x = 0.5$ 和 $t = 0.5$ 处的 WKB 解图像以及通过谱方法求解得到的解图像，都很好地反映了这一特征。同时，通过三维图像，我们也能直观地观察到半经典薛定谔方程的解同样表现出高频振荡的特性。①

进一步地，若以谱方法得到的解作为对比，可以发现 PINN 求解出的 WKB 解已经非常接近谱方法所求得的解，显示出较高的精度。同时，我们计算了二者差值的 L^2 范数，结果表明它们之间的整体差异非常小，尤其是在实部上的绝对误差也非常小。这表明，WKB 解的近似效果非常成功且有效。

但是也可能存在一些隐患, 比如:

- 对于振幅 a^ε 的近似只做到了第二项，$a^\varepsilon \approx a + a_1\varepsilon$ 并未进一步展开。如果进行更高阶展开，结果可能会更加精确
- 当前的训练次数和训练点数存在不足。如果进一步增加点数和训练次数，结果将更加完善
- 代码实现的细节仍有进一步优化的空间

① 具体使用 Python 和 PINN 求解半经典薛定谔方程的代码、利用谱方法求解的代码，以及最终绘制二者对比图的代码，均可在附录 A 中找到

3 采点分析

3.1 想法

根据上一章的计算以及实验，我们成功用 PINN 求出了半经典薛定谔方程的 WKB 解，目的是为了让我们在使用 PINN 求解薛定谔方程的时候，有一个大致的参照，我们先阐述一下，目前遇到的问题：首先求解薛定谔方程十分的重要，那么既然产生了使用 PINN 去求解薛定谔方程的方法，我们自然会去思考如何高效的求出薛定谔方程，在使用 PINN 去求解薛定谔方程的时候，一般总损失函数如下:

$$\mathcal{L}_{\text{Total}} = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}, \tag{3.1}$$

$$\text{其中,}\lambda_{\text{PDE}}, \lambda_{\text{IC}}, \lambda_{\text{BC}}\text{为参数,} \tag{3.2}$$

$$\mathcal{L}_{\text{PDE}}, \mathcal{L}_{\text{IC}}, \mathcal{L}_{\text{BC}}\text{分别为方程, 初始条件以及边界条件的损失函数} \tag{3.3}$$

给出半经典薛定谔方程：

$$i\varepsilon\partial_t u^\varepsilon = -\frac{\varepsilon^2}{2}\Delta u^\varepsilon + V(x)u^\varepsilon, u^\varepsilon(0,x) = u_{in}^\varepsilon(0,x) \tag{3.4}$$

$$\tag{3.5}$$

那么它的损失函数如下，

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{PDE}}}\sum_{i=1}^{N_{\text{PDE}}}\left|i\varepsilon\frac{\partial\psi_\theta(x_i,t_i)}{\partial t} + \frac{\varepsilon^2}{2}\frac{\partial^2\psi_\theta(x_i,t_i)}{\partial x^2} - V(x_i)\psi_\theta(x_i,t_i)\right|^2 \tag{3.6}$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}}\sum_{j=1}^{N_{\text{IC}}}\left|\psi_\theta(x_j,0) - \psi_0(x_j)\right|^2 \tag{3.7}$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{\text{BC}}}\sum_{k=1}^{N_{\text{BC}}}\left|\psi_\theta(x_k^{\text{BC}},t_k) - \psi_{\text{BC}}(x_k^{\text{BC}},t_k)\right|^2 \tag{3.8}$$

通常情况下，N 的取值往往非常大，与此同时，我们的研究主要关注 ε 趋近于 0 的极限情形。这种情形下，传统的训练点选取方法需要极其庞大的样本量。因此，我们渴望开发一种全新的采样策略，以显著提升神经网络训练效率，加速整体训练进程。以下将详细阐述我的研究探索:

3.2 取点策略

3.2.1 数值细节

在训练神经网络之前我们采取了如下的方式：

1. 神经网络结构（PINN）

神经网络由多个全连接层组成，层的配置由以下参数决定：

$$\text{layers} = [2, 64, 64, 64, 2]$$

输入为时间 t 和空间位置 x，输出为复值波函数 $u(t, x)$，其中实部和虚部分别对应网络的两部分输出。使用 Tanh 作为激活函数。

2. 薛定谔方程和残差

薛定谔方程的残差计算通过以下公式实现：

$$i\varepsilon \frac{\partial u}{\partial t} + \frac{\varepsilon^2}{2} \frac{\partial^2 u}{\partial x^2} - V(x)u = 0$$

其中 $u(t, x)$ 是波函数，ε 是某种物理参数，$V(x)$ 是势能函数。

3. 损失函数

损失函数由 PDE 残差和初始条件损失组成，总损失为：

$$\text{loss} = w_1 \cdot \text{loss}_{\text{PDE}} + w_2 \cdot \text{loss}_{\text{initial}}$$

其中：

$$w_1 = 0.9, \quad w_2 = 1 - w_1$$

损失函数中包括均匀采样的时间 t 和空间 x 上的节点。

4. 初始条件

初始条件的波函数为：

$$u(x, t = 0) = \exp\left(-\frac{(x - x_0)^2}{2\sigma^2}\right) \exp(ik_0 x)$$

其中 $x_0 = 0.0$, $k_0 = 2.0$, $\sigma = 0.5$ 是初始的高斯波包。

5. 势能函数

势能函数为简单的谐振势：

$$V(x) = \frac{1}{2}x^2$$

6. 训练参数

训练参数设置如下 (如无特殊说明，均为以下配置):

$$\varepsilon = 0.1, \quad N_{\text{collocation}} = 3000, \quad N_{\text{initial}} = 1000$$

学习率:

$$\text{learning rate} = 1 \times 10^{-3}, \quad \text{epochs} = 3000$$

优化器使用 Adam 优化器，训练过程通过最小化总损失函数来优化网络参数。

3.2.2 配置点以及个数

在其他条件相同的情况下，仅考虑配置点以及训练次数的情况下，损失函数的精度可以进一步提升，可见下表:

配置点数（$N_{\text{collocation}}$）	初始点数（N_{initial}）	训练次数（Epochs）	损失函数精度
1000	500	5000	1.5029×10^{-3}
2000	1000	5000	6.5974×10^{-4}
3000	1500	5000	5.3387×10^{-4}
1000	500	10000	1.1754×10^{-4}
2000	1000	10000	1.7600×10^{-4}
3000	1500	10000	2.2838×10^{-4}

表 3.1　不同配置点数、初始点数和训练次数下的损失函数精度

3.2.3 随机取点

① 在相同的条件下，我们不考虑任何的取点策略，而是直接采用随机采样的方式，得到的损失函数如下:

① 常规采样代码可以参考附录 B

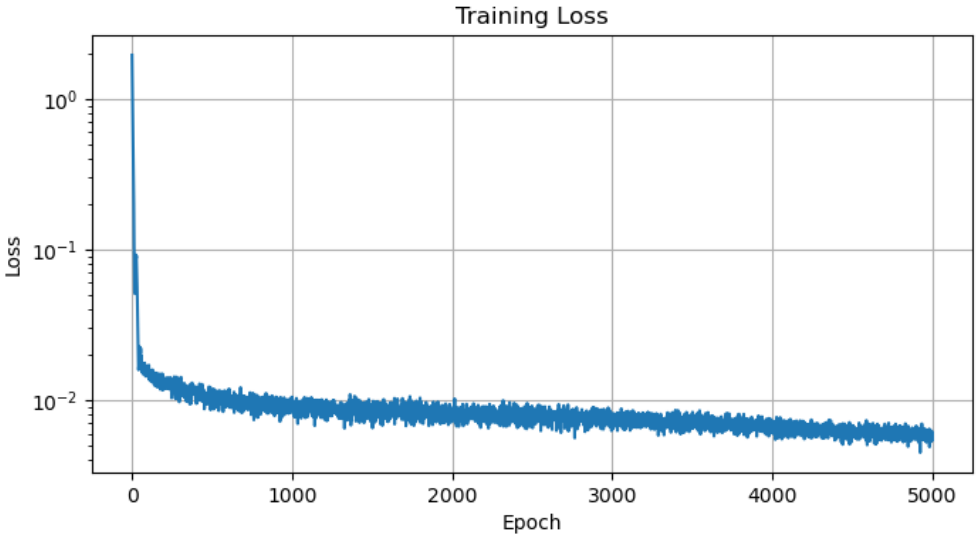


图 3.1　Loss Function: Random Sampling

采用正规采样的方式，得到的损失函数：

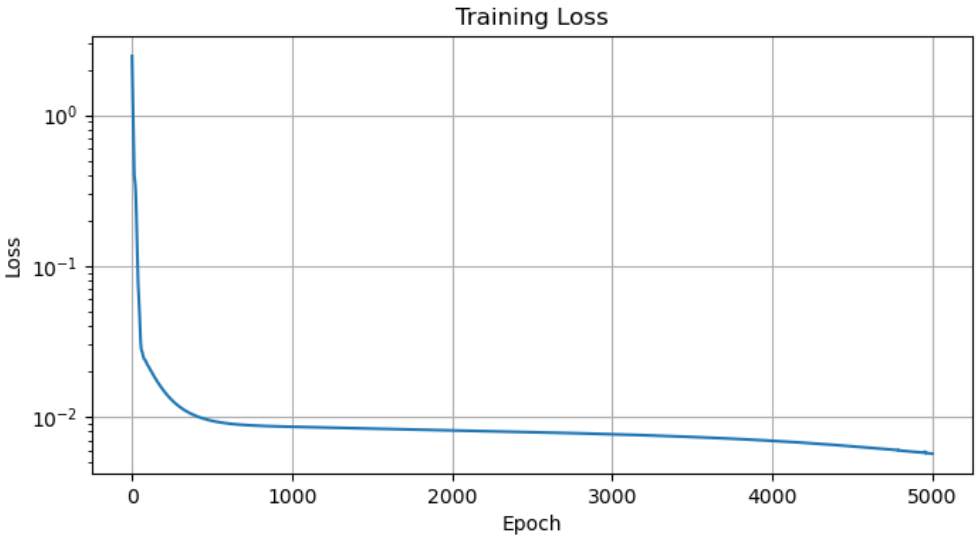


图 3.2　Loss Function: Normal Sampling

采用 Latin Hypercube Sampling，得到的损失函数

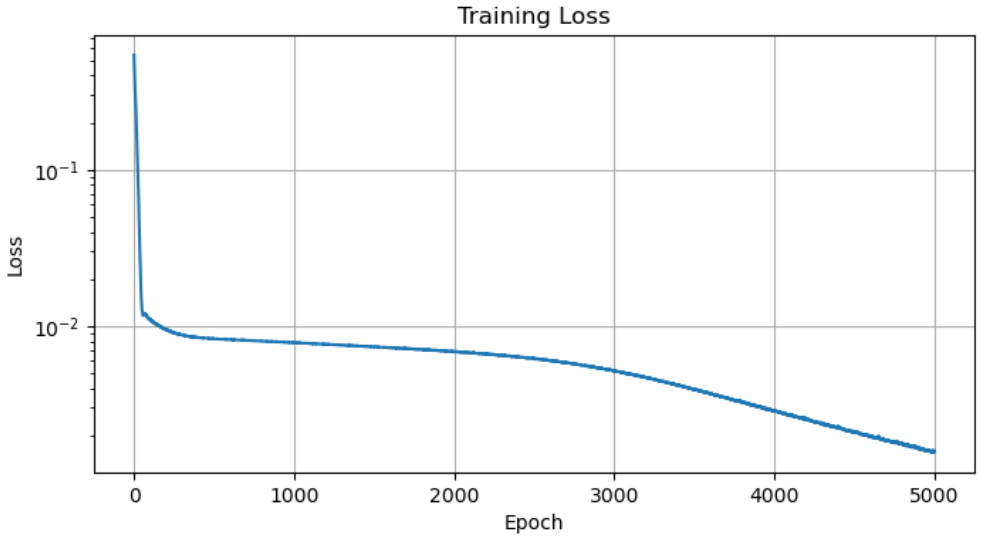


图 3.3　Loss Function: Latin Sampling

由于半经典薛定谔方程的解通常震荡较大而且可能具有周期性(一般根据$V(X)$是否具有周期性)，所以采取正规采样相对于随机采样，我们训练神经网络会更加轻松。

3.2.4　分析与总结

首先，我们发现正规采样比随机采样效果好。其次，在其他条件不变的情况下，增加配置点和初始条件的点数都能提高损失函数的精度。然而，尽管增加训练次数能够在一定程度上提高精度，过度训练往往会适得其反，导致损失函数精度的下降（参见表 3.1）。此外，结合[18] [26]及相关采样文献[19] [27]，现有的采样方法大致可以归纳为两类：一种是在初始采样时使用不同的采样策略，如随机采样、正规采样等，这些方法之间存在一定的差异，但差别并不十分显著，很难从本质上提高损失函数的精度，但是根据具体情况，选择更合适的初始采样策略，还是可以一定程度上提升训练效率；另一种是采用自适应采样、自适应加权或提升学习率等手段来提高损失函数的精度，但这些方法的适用范围较广，可以说，几乎适用于所有神经网络求解 PDE 的情况，但是，对于半经典薛定谔方程而言，并非是一种专门设计给半经典薛定谔方程方法，所以，效果上很难立竿见影。总体而言，若想提高半经典薛定谔方程的训练精度，我的理解是应侧重于初始采样策略，例如使用拉丁超立方采样（Latin Sampling）结合自适应采样等方法，这样可以有效提升训练效率和精度。

*

参考文献

[1] NIELSEN M A, CHUANG I L. Quantum computation and quantum information[M]. 10th ed. Cambridge University Press, 2010.

[2] BARDEEN J, COOPER L N, SCHRIEFFER J R. Theory of superconductivity[J]. Physical Review, 1957, 108(5): 1175-1204.

[3] PITAEVSKII L, STRINGARI S. Bose-einstein condensation and superfluidity[M]. Oxford University Press, 2016.

[4] GROSS E P. Structure of a quantized vortex in boson systems[J]. Il Nuovo Cimento, 1961, 20 (3): 454-477.

[5] DEUTSCH J M. Quantum statistical mechanics in a closed system[J]. Physical Review A, 1991, 43(4): 2046.

[6] BREUER H P, PETRUCCIONE F. The theory of open quantum systems[M]. Oxford University Press, 2007.

[7] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. Imagenet classification with deep convolutional neural networks[C]//Advances in Neural Information Processing Systems. 2012: 1097-1105.

[8] ALIPANAHI B, DELONG A, WEIRAUCH M T, et al. Predicting the sequence specificities of dna- and rna-binding proteins by deep learning[J]. Nature Biotechnology, 2015, 33(8): 831-838.

[9] LAKE B M, SALAKHUTDINOV R, TENENBAUM J B. Human-level concept learning through probabilistic program induction[J]. Science, 2015, 350(6266): 1332-1338.

[10] CARLES R. Semi-classical analysis for nonlinear schrödinger equations[J]. World Scientific, 2008.

[11] LEVEQUE R J. Finite difference methods for ordinary and partial differential equations[M]. Philadelphia: SIAM, 2007.

[12] EVANS L C. Graduate studies in mathematics: Vol. 19 partial differential equations[M]. American Mathematical Society, 2010.

[13] BRENNER S C, SCOTT L R. Texts in applied mathematics: Vol. 15 the mathematical theory of finite element methods[M]. Springer, 2008.

[14] JIN S, MARKOWICH P A, SPARBER C. Semiclassical models for the schrödinger equation with periodic potentials and band crossings[J]. Multiscale Modeling & Simulation, 2011, 9(4): 1371-1392.

[15] BAO W, JIN S, MARKOWICH P A. Numerical study of time-splitting spectral discretizations of nonlinear Schrödinger equations in the semiclassical regimes[J/OL]. SIAM Journal on Numerical Analysis, 2003, 41(1): 27-64. https://doi.org/10.1137/S1064827501393253.

[16] RAISSI M, PERDIKARIS P, KARNIADAKIS G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differ-

ential equations[J]. Journal of Computational Physics, 2019, 378: 686-707.

[17] BAYDIN A G, PEARLMUTTER B A, RADUL A A, et al. Automatic differentiation in machine learning: a survey[J/OL]. Journal of Machine Learning Research, 2018, 18(153): 1-43. http://jmlr.org/papers/v18/17-468.html.

[18] WU C, ZHU M, TAN Q, et al. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks[A/OL]. 2022. arXiv: 2207.10289. https://doi.org/10.48550/arXiv.2207.10289.

[19] WANG T, BAI Z, ZHAO H. Effects of sampling strategies and optimization methods on solving wave equation with physics-informed neural network[M]. Xi'an, Shaanxi, 710049, China: School of Mathematics and Statistics, Xi'an Jiaotong University, 2023.

[20] BREVI L, MANDARINO A, PRATI E. A tutorial on the use of physics-informed neural networks to compute the spectrum of quantum systems[J]. Tutorial, 2023.

[21] CHAI L, JIN S, LI Q. Semiclassical models for the schrödinger equation with periodic potentials and band crossings[J/OL]. SIAM Journal on Mathematical Analysis, 2020, 52(2): 1219-1245. DOI: 10.1137/19M1263459.

[22] SHI JIN C S, Peter Markowich. Mathematical and computational methods for semiclassical schrödinger equations[J/OL]. Acta Numerica, 2011: 121-209. DOI: 10.1017/S0962492911000031.

[23] DRISCOLL T A, HALE N, TREFETHEN L N. Chebfun guide[M/OL]. Chebfun, 2014. http://www.chebfun.org.

[24] ISERLES A. A first course in the numerical analysis of differential equations[M/OL]. 2nd ed. Cambridge University Press, 2009. https://books.google.com/books?id=HBgQAQAAIAAJ.

[25] SAKURAI J J, NAPOLITANO J. Modern quantum mechanics[M]. 2nd ed. Cambridge University Press, 2017.

[26] FLORIDO J, WANG H, KHAN A, et al. Investigating guiding information for adaptive collocation point sampling in pinns[J]. Journal Name.

[27] BAUDUIN V, CUOMO S, COLA V S D. Impact of collocation point sampling techniques on pinn performance in groundwater flow predictions[M]. Via Cintia, 26, Napoli, 80126, Italy: Department of Mathematics and Applications "R. Caccioppoli", University of Naples Federico II, 2023.

附录 A WKB 求解代码

```
import torch
import torch.nn as nn
import torch.autograd as autograd
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cm
import math

# 定义PINN模型
class PINN(nn.Module):
    def __init__(self, layers):
        super(PINN, self).__init__()
        self.net = nn.Sequential()
        for i in range(len(layers) - 2):
        self.net.add_module(f"layer_{i}", nn.Linear(layers[i], layers[i+1]))
            self.net.add_module(f"tanh_{i}", nn.Tanh())
       self.net.add_module("output", nn.Linear(layers[-2], layers[-1]))

    def forward(self, x, t):
        input = torch.cat([x, t], dim=1)
        return self.net(input)

# 势能函数 V(x)
def V(x):
    return 0.5 * x**2  # 示例: 简谐势

# 初始条件 S_in(x)
def S_in(x):
    return torch.sin(x)  # 示例初始条件
```

```
# 计算PDE残差
def pde_residual(model, x, t):
    x.requires_grad_(True)
    t.requires_grad_(True)

    S = model(x, t)
  dS_dt = autograd.grad(S, t, grad_outputs=torch.ones_like(S), retain_graph=True, creat
  dS_dx = autograd.grad(S, x, grad_outputs=torch.ones_like(S), retain_graph=True, creat

    residual = dS_dt + 0.5 * dS_dx**2 + V(x)
    return residual

# 训练模型
def train(model, optimizer, epochs, collocation_points, initial_points):
    for epoch in range(epochs):
        optimizer.zero_grad()

        # PDE残差损失
        x_f, t_f = collocation_points
        res = pde_residual(model, x_f, t_f)
        loss_f = torch.mean(res**2)

        # 初始条件损失
        x_0, t_0 = initial_points
        S_pred_0 = model(x_0, t_0)
        S_true_0 = S_in(x_0)
        loss_0 = torch.mean((S_pred_0 - S_true_0)**2)

        # 总损失
        loss = loss_f + loss_0
        loss.backward()
        optimizer.step()
```

```
        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss.item():.6f}")

# 创建模型
layers = [2, 64, 64, 64, 1]
model = PINN(layers)

# 优化器
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# 训练数据点（可以扩展更密集或更大的范围）
N_f = 1000  # collocation points
N_0 = 200  # initial condition points

x_f = torch.rand(N_f, 1) * 2 - 1  # [-1, 1]
t_f = torch.rand(N_f, 1) * 1  # [0, 1]

x_0 = torch.rand(N_0, 1) * 2 - 1  # [-1, 1]
t_0 = torch.zeros_like(x_0)  # t = 0

# 训练
train(model, optimizer, epochs=3000,
      collocation_points=(x_f, t_f),
      initial_points=(x_0, t_0))


# 冻结已训练好的S模型的参数，防止在训练a时被修改
for param in model.parameters():
    param.requires_grad = False

# 定义新的PINN模型用于求解a
class PINN_a(nn.Module):
    def __init__(self, layers):
```

```
        super(PINN_a, self).__init__()
        self.net = nn.Sequential()
        for i in range(len(layers)-2):
        self.net.add_module(f"layer_{i}", nn.Linear(layers[i], layers[i+1]))
            self.net.add_module(f"tanh_{i}", nn.Tanh())
       self.net.add_module("output", nn.Linear(layers[-2], layers[-1]))

    def forward(self, x, t):
        inputs = torch.cat([x, t], dim=1)
        return self.net(inputs)

# 定义初始条件a_in(x)
def a_in(x):
    return torch.exp(-x**2)

# 计算a的PDE残差
def pde_residual_a(model_a, model_S, x, t):
    x.requires_grad_(True)
    t.requires_grad_(True)

    # 计算a及其导数
    a = model_a(x, t)
    da_dt = autograd.grad(a, t, grad_outputs=torch.ones_like(a),
                          retain_graph=True, create_graph=True)[0]
    da_dx = autograd.grad(a, x, grad_outputs=torch.ones_like(a),
                          retain_graph=True, create_graph=True)[0]

    # 计算S的导数（使用已冻结的S模型）
    with torch.enable_grad():  # 确保梯度计算开启
        S = model_S(x, t)
        dS_dx = autograd.grad(S, x, grad_outputs=torch.ones_like(S),
                              retain_graph=True, create_graph=True)[0]
     d2S_dx2 = autograd.grad(dS_dx, x, grad_outputs=torch.ones_like(dS_dx),
```

```
                                retain_graph=True, create_graph=True)[0]

    # 计算PDE残差
    residual = da_dt + dS_dx*da_dx + 0.5*a*d2S_dx2
    return residual

# 修改后的训练函数
def train_a(model_a, model_S, optimizer, epochs, collocation_points, initial_points):
    for epoch in range(epochs):
        optimizer.zero_grad()

        # PDE残差损失
        x_f, t_f = collocation_points
        residual = pde_residual_a(model_a, model_S, x_f, t_f)
        loss_f = torch.mean(residual**2)

        # 初始条件损失
        x_0, t_0 = initial_points
        a_pred = model_a(x_0, t_0)
        a_true = a_in(x_0)
        loss_0 = torch.mean((a_pred - a_true)**2)

        # 总损失
        total_loss = loss_f + loss_0
        total_loss.backward()
        optimizer.step()

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {total_loss.item():.6f}")

# 创建并训练a模型
layers_a = [2, 64, 64, 64, 1]
model_a = PINN_a(layers_a)
```

```
optimizer_a = torch.optim.Adam(model_a.parameters(), lr=1e-3)

# 生成训练数据（可根据需要调整范围和数量）
N_f_a = 2000  # 配点数量
N_0_a = 500   # 初始条件点数

x_f_a = torch.rand(N_f_a, 1)*2 - 1  # x □ [-1, 1]
t_f_a = torch.rand(N_f_a, 1)        # t □ [0, 1]

x_0_a = torch.rand(N_0_a, 1)*2 - 1  # 初始时刻x分布
t_0_a = torch.zeros_like(x_0_a)     # 初始时刻t=0

# 开始训练
train_a(model_a, model, optimizer_a, epochs=3000,
        collocation_points=(x_f_a, t_f_a),
        initial_points=(x_0_a, t_0_a))



# 冻结已有模型参数

for param in model.parameters():    # 冻结S模型
    param.requires_grad = False
for param in model_a.parameters(): # 冻结a模型（用于计算□a）
    param.requires_grad = False

# ======================
# 定义新的a1模型（网络结构保持不变）
# ======================
class PINN_a1(nn.Module):
    def __init__(self, layers):
        super(PINN_a1, self).__init__()
        self.net = nn.Sequential()
        for i in range(len(layers)-2):
```

```
        self.net.add_module(f"layer_{i}a1", nn.Linear(layers[i], layers[i+1]))
            self.net.add_module(f"tanh_{i}a1", nn.Tanh())
        self.net.add_module("output_a1", nn.Linear(layers[-2], layers[-
1]))

    def forward(self, x, t):
        inputs = torch.cat([x, t], dim=1)
        return self.net(inputs)


# ======================
# 修改PDE残差计算（关键修正点）
# ======================
def pde_residual_a1(model_a1, model_S, model_a, x, t): # 新增model_a参数
    x.requires_grad_(True)
    t.requires_grad_(True)

    # 计算a1及其导数
    a1 = model_a1(x, t)
    da1_dt = autograd.grad(a1, t, grad_outputs=torch.ones_like(a1),
                           retain_graph=True, create_graph=True)[0]
    da1_dx = autograd.grad(a1, x, grad_outputs=torch.ones_like(a1),
                           retain_graph=True, create_graph=True)[0]

    # 计算S的导数
    with torch.enable_grad():
        S = model_S(x, t)
        dS_dx = autograd.grad(S, x, grad_outputs=torch.ones_like(S),
                              retain_graph=True, create_graph=True)[0]
     d2S_dx2 = autograd.grad(dS_dx, x, grad_outputs=torch.ones_like(dS_dx),
                                retain_graph=True, create_graph=True)[0]

    # 关键修正：计算□a（来自已冻结的a模型）
```

```
    with torch.enable_grad():
        a = model_a(x, t)
        da_dx = autograd.grad(a, x, grad_outputs=torch.ones_like(a),
                              retain_graph=True, create_graph=True)[0]
      d2a_dx2 = autograd.grad(da_dx, x, grad_outputs=torch.ones_like(da_dx),
                                retain_graph=True, create_graph=True)[0]

    # 残差公式修正
   residual = da1_dt + dS_dx*da1_dx + 0.5*a1*d2S_dx2 - d2a_dx2  # 末项改为□a
    return residual

# 定义a1的初始条件
def a1_in(x):
    return torch.sin(x)

# ======================
# 修改训练函数
# ======================
def train_a1(model_a1, model_S, model_a, optimizer, epochs, collocation_points, initial_
    for epoch in range(epochs):
        optimizer.zero_grad()

        # PDE残差损失
        x_f, t_f = collocation_points
      residual = pde_residual_a1(model_a1, model_S, model_a, x_f, t_f)  # 传入model_a
        loss_f = torch.mean(residual**2)

        # 初始条件损失
        x_0, t_0 = initial_points
        a1_pred = model_a1(x_0, t_0)
        a1_true = a1_in(x_0)
        loss_0 = torch.mean((a1_pred - a1_true)**2)
```

```
        # 总损失
        total_loss = loss_f + loss_0
        total_loss.backward()
        optimizer.step()

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {total_loss.item():.6f}")

# ======================
# 创建并训练a1模型
# ======================
layers_a1 = [2, 64, 64, 64, 1]
model_a1 = PINN_a1(layers_a1)
optimizer_a1 = torch.optim.Adam(model_a1.parameters(), lr=1e-3)

# 生成训练数据（扩大采样范围）
N_f_a1 = 3000
N_0_a1 = 800

x_f_a1 = torch.rand(N_f_a1, 1)*2 - 1  # x □ [-1, 1]
t_f_a1 = torch.rand(N_f_a1, 1)        # t □ [0, 1]

x_0_a1 = torch.rand(N_0_a1, 1)*2 - 1  # 初始时刻x分布
t_0_a1 = torch.zeros_like(x_0_a1)     # 初始时刻t=0

# 开始训练（传入model_a）
train_a1(model_a1, model, model_a, optimizer_a1, epochs=3000,
        collocation_points=(x_f_a1, t_f_a1),
        initial_points=(x_0_a1, t_0_a1))



# 定义 epsilon
```

```
epsilon = 0.01

# 计算 WKB 解
def compute_wkb(model_a, model_a1, model_S, x, t, epsilon):
    # 计算 a(t, x), a_1(t, x) 和 S(t, x)
    a = model_a(x, t).detach().numpy()
    a1 = model_a1(x, t).detach().numpy()
    S = model_S(x, t).detach().numpy()

    # 计算 WKB 解
    amplitude = a + epsilon * a1  # 振幅部分
    phase = np.exp(1j * S / epsilon)  # 相位部分
    wkb = amplitude * phase  # WKB 解
    return wkb

# 绘制 WKB 解随 x 的变化 (固定 t)
def plot_wkb_fixed_t(model_a, model_a1, model_S, t_fixed, epsilon, x_range=(-
1, 1), num_points=100):
   x = torch.linspace(x_range[0], x_range[1], num_points).reshape(-1, 1)
    t = torch.full_like(x, t_fixed)

    # 计算 WKB 解
    wkb = compute_wkb(model_a, model_a1, model_S, x, t, epsilon)

    # 绘制实部和虚部
    plt.figure(figsize=(10, 6))
    plt.plot(x.numpy(), np.real(wkb), label="Re(u)")
    plt.plot(x.numpy(), np.imag(wkb), label="Im(u)")
    plt.xlabel("x")
    plt.ylabel("u")
    plt.title(f"WKB Solution at t={t_fixed}, epsilon={epsilon}")
    plt.legend()
    plt.grid()
```

```
    plt.show()

# 绘制 WKB 解随 t 的变化（固定 x）
def plot_wkb_fixed_x(model_a, model_a1, model_S, x_fixed, epsilon, t_range=(0, 1), num_p
    t = torch.linspace(t_range[0], t_range[1], num_points).reshape(-1, 1)
    x = torch.full_like(t, x_fixed)

    # 计算 WKB 解
    wkb = compute_wkb(model_a, model_a1, model_S, x, t, epsilon)

    # 绘制实部和虚部
    plt.figure(figsize=(10, 6))
    plt.plot(t.numpy(), np.real(wkb), label="Re(u)")
    plt.plot(t.numpy(), np.imag(wkb), label="Im(u)")
    plt.xlabel("t")
    plt.ylabel("u")
    plt.title(f"WKB Solution at x={x_fixed}, epsilon={epsilon}")
    plt.legend()
    plt.grid()
    plt.show()

# 绘制 WKB 解的二维图像（实部或虚部）
def plot_wkb_2d(model_a, model_a1, model_S, epsilon, x_range=(-1, 1), t_range=(0, 1), nu
    x = torch.linspace(x_range[0], x_range[1], num_points)
    t = torch.linspace(t_range[0], t_range[1], num_points)
    X, T = torch.meshgrid(x, t)
    X_flat = X.reshape(-1, 1)
    T_flat = T.reshape(-1, 1)

    # 计算 WKB 解
    wkb = compute_wkb(model_a, model_a1, model_S, X_flat, T_flat, epsilon)
    wkb = wkb.reshape(X.shape)
```

```
    # 选择绘制实部或虚部
    if plot_component == "real":
        data = np.real(wkb)
        title = "Re(u)"
    elif plot_component == "imag":
        data = np.imag(wkb)
        title = "Im(u)"
    else:
        raise ValueError("plot_component must be 'real' or 'imag'")

    # 绘制二维图像
    fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
   surf = ax.plot_surface(X.numpy(), T.numpy(), data, cmap=cm.coolwarm, linewidth=0, ant
    ax.set_xlabel("x")
    ax.set_ylabel("t")
    ax.set_zlabel(title)
    ax.set_title(f"WKB Solution {title}, epsilon={epsilon}")
    fig.colorbar(surf, shrink=0.5, aspect=5)
    plt.show()

# 固定 t=0.5, 绘制 WKB 解随 x 的变化
plot_wkb_fixed_t(model_a, model_a1, model, t_fixed=0.5, epsilon=epsilon)

# 固定 x=0.5, 绘制 WKB 解随 t 的变化
plot_wkb_fixed_x(model_a, model_a1, model, x_fixed=0.5, epsilon=epsilon)

# 绘制 WKB 解的二维图像 (实部)
plot_wkb_2d(model_a, model_a1, model, epsilon, plot_component="real")

# 绘制 WKB 解的二维图像 (虚部)
plot_wkb_2d(model_a, model_a1, model, epsilon, plot_component="imag")

import numpy as np
```

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 参数设置
epsilon = 0.01
Xmax = 20.0
Nx = 2**12
dx = 2 * Xmax / Nx
x = np.linspace(-Xmax, Xmax, Nx, endpoint=False)
u0 = (epsilon * np.sin(x) + np.exp(-x**2)) * np.exp(1j * np.sin(x)/epsilon)
V = 0.5 * x**2
T_total = 1.0  # 调整总时间为1, 使t□[0,1]
dt = 0.01
Nt = int(T_total / dt)
t_values = np.linspace(0, T_total, Nt)
k = 2 * np.pi * np.fft.fftfreq(Nx, dx)
exp_K = np.exp(-1j * epsilon * dt * (k**2) / 2)
exp_V_half = np.exp(-1j * V * dt / (2 * epsilon))
u_history = np.zeros((Nt, Nx), dtype=np.complex128)

# 时间演化
psi = u0.copy()
for i in range(Nt):
    psi *= exp_V_half
    psi_hat = np.fft.fft(psi)
    psi_hat *= exp_K
    psi = np.fft.ifft(psi_hat)
    psi *= exp_V_half
    u_history[i, :] = psi

# 1. 固定x=0.5, 绘制u(0.5,t)的实部和虚部 (t□[0,1])
x_target = 0.5
x_idx = np.argmin(np.abs(x - x_target))
```

```
u_at_x_fixed = u_history[:, x_idx]

plt.figure(figsize=(10, 5))
plt.plot(t_values, np.real(u_at_x_fixed), 'r-', label='Re[u(0.5,t)]', linewidth=1.5)
plt.plot(t_values, np.imag(u_at_x_fixed), 'b--', label='Im[u(0.5,t)]', linewidth=1.5)
plt.xlim(0, 1)  # 固定t范围
plt.xlabel('Time (t)')
plt.ylabel('u(0.5,t)')
plt.title(f'Spectral Method at x = 0.5, epsilon = 0.01')
plt.grid(True)
plt.legend()
plt.show()

# 2. 固定t=0.5, 绘制u(x,0.5)的实部和虚部 (x□[-1,1])
t_target = 0.5
t_idx = np.argmin(np.abs(t_values - t_target))
u_at_t_fixed = u_history[t_idx, :]
x_mask = (x >= -1) & (x <= 1)  # 固定x范围

plt.figure(figsize=(10, 5))
plt.plot(x[x_mask], np.real(u_at_t_fixed)[x_mask], 'r-', label='Re[u(x,0.5)]', linewidt
plt.plot(x[x_mask], np.imag(u_at_t_fixed)[x_mask], 'b--', label='Im[u(x,0.5)]', linewid
plt.xlim(-1, 1)  # 显式设置x范围
plt.xlabel('x')
plt.ylabel('u(x,0.5)')
plt.title(f'Spectral Method at t = 0.5, epsilon = 0.01')
plt.grid(True)
plt.legend()
plt.show()

# 3. 三维时空演化图 (x□[-1,1], t□[0,1])
x_mask = (x >= -1) & (x <= 1)
t_mask = (t_values >= 0) & (t_values <= 1)  # 固定t范围
```

```
X, T = np.meshgrid(x[x_mask], t_values[t_mask])
Z = np.real(u_history[t_mask, :][:, x_mask])  # 双重索引确保范围正确

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, T, Z, cmap='viridis', rstride=5, cstride=5, alpha=0.8)
ax.set_xlabel('x')
ax.set_ylabel('t')
ax.set_zlabel('Re[u(x,t)]')
ax.set_title('Spectral Method 3D Evolution of Re[u(x,t)]')
plt.show()

# 新增代码: 绘制复数解u(x,t)的模和相位的3D图像
x_mask = (x >= -1) & (x <= 1)
t_mask = (t_values >= 0) & (t_values <= 1)
X, T = np.meshgrid(x[x_mask], t_values[t_mask])
U = u_history[t_mask, :][:, x_mask]  # 提取x□[-1,1], t□[0,1]的复数解

# 1. 绘制模 |u(x,t)| 的3D图
fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, T, np.abs(U), cmap='plasma', rstride=5, cstride=5, alpha=0.8)
ax1.set_xlabel('x')
ax1.set_ylabel('t')
ax1.set_zlabel('|u(x,t)|')
ax1.set_title('Spectral Method Magnitude |u(x,t)|')

# 2. 绘制相位 angle(u(x,t)) 的3D图
ax2 = fig.add_subplot(122, projection='3d')
phase = np.angle(U)  # 计算相位（弧度制）
ax2.plot_surface(X, T, phase, cmap='hsv', rstride=5, cstride=5, alpha=0.8)
ax2.set_xlabel('x')
ax2.set_ylabel('t ')
```

```
ax2.set_zlabel('Phase [rad]')
ax2.set_title('Spectral Method Phase angle(u(x,t)) ')
plt.tight_layout()
plt.show()

# 对比分析

# 定义对比参数
epsilon = 0.01
x_compare = 0.5  # 固定x值
t_compare = 0.5  # 固定t值
num_points = 200  # 插值点数

# 生成统一的时空网格（与谱方法数据对齐）
x_spec = np.linspace(-1, 1, Nx)[(x >= -1) & (x <= 1)]  # 谱方法在[-1,1]内的x坐标
t_spec = t_values  # 时间网格

# 1. 固定x=0.5，对比时间演化 ----------------------------------------
# 获取谱方法数据
spec_t = t_spec
spec_real = np.real(u_history[:, x_idx])
spec_imag = np.imag(u_history[:, x_idx])
spec_abs = np.abs(u_history[:, x_idx])

# 计算PINN解
x_pinn = torch.full((len(t_spec),1), x_compare, dtype=torch.float32)
t_pinn = torch.tensor(t_spec, dtype=torch.float32).reshape(-1,1)
wkb_pinn = compute_wkb(model_a, model_a1, model, x_pinn, t_pinn, epsilon)
pinn_real = np.real(wkb_pinn).flatten()
pinn_imag = np.imag(wkb_pinn).flatten()
pinn_abs = np.abs(wkb_pinn).flatten()
```

```
# 计算差值
diff_real = spec_real - pinn_real
diff_imag = spec_imag - pinn_imag
diff_abs = spec_abs - pinn_abs

# 绘图
plt.figure(figsize=(15, 10))

# 实部对比
plt.subplot(3, 1, 1)
plt.plot(spec_t, spec_real, 'r-', label='Spectral Re(u)', linewidth=2, alpha=0.7)
plt.plot(spec_t, pinn_real, 'b--', label='WKB Re(u)', linewidth=2, alpha=0.7)
plt.plot(spec_t, diff_real, 'g:', label='Difference', linewidth=2)
plt.xlabel('Time')
plt.ylabel('Value')
plt.title(f'Real Part Comparison at x={x_compare}')
plt.grid(True)
plt.legend()

# 虚部对比
plt.subplot(3, 1, 2)
plt.plot(spec_t, spec_imag, 'r-', label='Spectral Im(u)', linewidth=2, alpha=0.7)
plt.plot(spec_t, pinn_imag, 'b--', label='WKB Im(u)', linewidth=2, alpha=0.7)
plt.plot(spec_t, diff_imag, 'g:', label='Difference', linewidth=2)
plt.xlabel('Time')
plt.ylabel('Value')
plt.title(f'Imaginary Part Comparison at x={x_compare}')
plt.grid(True)
plt.legend()

# 模对比
plt.subplot(3, 1, 3)
```

```
plt.plot(spec_t, spec_abs, 'r-', label='Spectral |u|', linewidth=2, alpha=0.7)
plt.plot(spec_t, pinn_abs, 'b--', label='WKB |u|', linewidth=2, alpha=0.7)
plt.plot(spec_t, diff_abs, 'g:', label='Difference', linewidth=2)
plt.xlabel('Time')
plt.ylabel('Magnitude')
plt.title(f'Absolute Value Comparison at x={x_compare}')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

# 2. 固定t=0.5, 对比空间分布 ----------------------------------------
-----------------------
# 获取谱方法数据
spec_x = x_spec
spec_real = np.real(u_history[t_idx, (x >= -1) & (x <= 1)])
spec_imag = np.imag(u_history[t_idx, (x >= -1) & (x <= 1)])
spec_abs = np.abs(u_history[t_idx, (x >= -1) & (x <= 1)])

# 计算PINN解
t_pinn = torch.full((len(x_spec),1), t_compare, dtype=torch.float32)
x_pinn = torch.tensor(x_spec, dtype=torch.float32).reshape(-1,1)
wkb_pinn = compute_wkb(model_a, model_a1, model, x_pinn, t_pinn, epsilon)
pinn_real = np.real(wkb_pinn).flatten()
pinn_imag = np.imag(wkb_pinn).flatten()
pinn_abs = np.abs(wkb_pinn).flatten()

# 计算差值
diff_real = spec_real - pinn_real
diff_imag = spec_imag - pinn_imag
diff_abs = spec_abs - pinn_abs
```

```
# 绘图
plt.figure(figsize=(15, 10))

# 实部对比
plt.subplot(3, 1, 1)
plt.plot(spec_x, spec_real, 'r-', label='Spectral Re(u)', linewidth=2, alpha=0.7)
plt.plot(spec_x, pinn_real, 'b--', label='WKB Re(u)', linewidth=2, alpha=0.7)
plt.plot(spec_x, diff_real, 'g:', label='Difference', linewidth=2)
plt.xlabel('Position')
plt.ylabel('Value')
plt.title(f'Real Part Comparison at t={t_compare}')
plt.grid(True)
plt.legend()

# 虚部对比
plt.subplot(3, 1, 2)
plt.plot(spec_x, spec_imag, 'r-', label='Spectral Im(u)', linewidth=2, alpha=0.7)
plt.plot(spec_x, pinn_imag, 'b--', label='WKB Im(u)', linewidth=2, alpha=0.7)
plt.plot(spec_x, diff_imag, 'g:', label='Difference', linewidth=2)
plt.xlabel('Position')
plt.ylabel('Value')
plt.title(f'Imaginary Part Comparison at t={t_compare}')
plt.grid(True)
plt.legend()

# 模对比
plt.subplot(3, 1, 3)
plt.plot(spec_x, spec_abs, 'r-', label='Spectral |u|', linewidth=2, alpha=0.7)
plt.plot(spec_x, pinn_abs, 'b--', label='WKB |u|', linewidth=2, alpha=0.7)
plt.plot(spec_x, diff_abs, 'g:', label='Difference', linewidth=2)
plt.xlabel('Position')
plt.ylabel('Magnitude')
plt.title(f'Absolute Value Comparison at t={t_compare}')
```

```
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

# 范数计算

def compute_norms(true_sol, pred_sol, delta):
    error = true_sol - pred_sol
    l1 = np.sum(np.abs(error)) * delta
    l2 = np.sqrt(np.sum(error**2) * delta)
    l_inf = np.max(np.abs(error))
    return {'L1': l1, 'L2': l2, 'L_infty': l_inf}

# 1. 固定x=0.5的时间演化范数 ----------------------------------------
----------
# 参数
delta_t = t_spec[1] - t_spec[0]  # 时间步长

# 计算范数
norms_time = compute_norms(spec_real + 1j*spec_imag, pinn_real + 1j*pinn_imag, delta_t)

# 打印结果
print("\n固定 x=0.5 的时间演化误差范数 (t□[0,1]):")
print(f"L² 范数: {norms_time['L2']:.4e}")


# 2. 固定t=0.5的空间分布范数 ----------------------------------------
---------
# 计算空间步长 (在谱方法网格的[-1,1]区间内)
x_spec_subset = x[(x >= -1) & (x <= 1)]
delta_x = x_spec_subset[1] - x_spec_subset[0]  # 空间步长
```

```
# 计算范数
norms_space = compute_norms(spec_real + 1j*spec_imag, pinn_real + 1j*pinn_imag, delta_x)

# 打印结果
print("\n固定 t=0.5 的空间分布误差范数 (x□[-1,1]):")
print(f"L² 范数: {norms_space['L2']:.4e}")
```

附录 B 常规采样分析代码

```
    import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from torch.autograd import grad

# ==================== 1. 定义神经网络 ====================
class PINN(nn.Module):
    def __init__(self, layers):
        super(PINN, self).__init__()
        self.linear_layers = nn.ModuleList()
        for i in range(len(layers) - 1):
          self.linear_layers.append(nn.Linear(layers[i], layers[i + 1]))
            if i < len(layers) - 2:
              self.linear_layers.append(nn.Tanh())  # 使用 Tanh 激活函数

    def forward(self, t, x):
        inputs = torch.cat([t, x], dim=1)
        for layer in self.linear_layers:
            inputs = layer(inputs)
      return inputs[:, 0:1] + 1j * inputs[:, 1:2] # 输出复数 (实部 + 虚部)

# ==================== 2. 定义薛定谔方程和损失函数 ====================
def schrodinger_residual(u, t, x, epsilon, V):
    # 计算梯度
   u_t = grad(u, t, grad_outputs=torch.ones_like(u), create_graph=True)[0]
   u_x = grad(u, x, grad_outputs=torch.ones_like(u), create_graph=True)[0]
   u_xx = grad(u_x, x, grad_outputs=torch.ones_like(u_x), create_graph=True)[0]

    # 计算残差
```

```
    residual = 1j * epsilon * u_t + (epsilon**2 / 2) * u_xx - V(x) * u
    return residual

def compute_loss(model, epsilon, V, t_domain, x_domain, N_collocation, N_initial):
    w_1 = 0.9
    w_2 =1 - w_1
    # 1. 均匀采样 PDE 残差点 (t, x) □ [0, T] × [x_min, x_max]
  t_coll = torch.linspace(t_domain[0], t_domain[1], int(N_collocation**0.5)).view(-
1, 1)  # 时间均匀采样
  x_coll = torch.linspace(x_domain[0], x_domain[1], int(N_collocation**0.5)).view(-
1, 1)  # 空间均匀采样
  T_grid, X_grid = torch.meshgrid(t_coll.squeeze(), x_coll.squeeze(), indexing='xy')
    t_coll = T_grid.reshape(-1, 1)  # 展平为 (N_collocation, 1)
    x_coll = X_grid.reshape(-1, 1)  # 展平为 (N_collocation, 1)
    t_coll.requires_grad_(True)
    x_coll.requires_grad_(True)

    u_pred = model(t_coll, x_coll)
    residual = schrodinger_residual(u_pred, t_coll, x_coll, epsilon, V)
    loss_pde = torch.mean(torch.abs(residual)**2)

    # 2. 初始条件均匀采样 (t=0)
  x_initial = torch.linspace(x_domain[0], x_domain[1], N_initial).view(-
1, 1)  # x □ [x_min, x_max]
    u_initial_true = u_in_initial_condition(x_initial)
    u_initial_pred = model(torch.zeros_like(x_initial), x_initial)
  loss_initial = torch.mean(torch.abs(u_initial_pred - u_initial_true)**2)

    # 3. 总损失
    total_loss = w_1*loss_pde + w_2*loss_initial
    return total_loss

# ==================== 3. 初始条件（待定部分） ====================
```

```
def u_in_initial_condition(x):
    x0 = 0.0  # 初始中心
    k0 = 2.0  # 初始动量
    sigma = 0.5  # 宽度
   return torch.exp(-(x - x0)**2 / (2 * sigma**2)) * torch.exp(1j * k0 * x)

# ==================== 4. 定义势能函数 ====================
def V(x):
    return 0.5 * x**2  # 简谐势 (可替换)

# ==================== 5. 训练参数和模型初始化 ====================
# 定义域
t_domain = [0.0, 1.0]  # t □ [0, T]
x_domain = [-5.0, 5.0]  # x □ [x_min, x_max]

# 神经网络结构
layers = [2, 64, 64, 64, 2]  # 输入 (t, x), 输出 (Re(u), Im(u))

# 训练参数
epsilon = 0.1  # 小参数
N_collocation = 3000  # PDE 残差点数量
N_initial = 1000      # 初始条件点数量
learning_rate = 1e-4
epochs = 5000

# 初始化模型和优化器
model = PINN(layers)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate, weight_decay=1e-
5,betas=(0.9, 0.999))

# ==================== 6. 训练循环 ====================
loss_history = []
for epoch in range(epochs):
```

```
    optimizer.zero_grad()
   loss = compute_loss(model, epsilon, V, t_domain, x_domain, N_collocation, N_initial)
    loss.backward()
    optimizer.step()
    loss_history.append(loss.item())

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4e}")

# ==================== 7. 可视化结果 ====================
# 绘制损失曲线
plt.figure(figsize=(8, 4))
plt.semilogy(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss")
plt.grid()
plt.show()

# 绘制波函数的实部和概率密度
t_test = torch.linspace(t_domain[0], t_domain[1], 100)
x_test = torch.linspace(x_domain[0], x_domain[1], 100)
T, X = torch.meshgrid(t_test, x_test, indexing='xy')

# 将 T 和 X 转换为 NumPy 数组
T_np = T.cpu().numpy()
X_np = X.cpu().numpy()

# 计算预测值并转换为 NumPy
u_pred = model(T.reshape(-1, 1), X.reshape(-1, 1)).detach()
u_pred_np = u_pred.cpu().numpy().reshape(100, 100)

# 实部
```

```
plt.figure(figsize=(10, 6))
plt.pcolormesh(T_np, X_np, np.real(u_pred_np), shading='auto', cmap='viridis')
plt.colorbar(label="Re(u)")
plt.xlabel("t")
plt.ylabel("x")
plt.title("Real part of the wave function")
plt.show()

#等高线图 (Contour Plot)
plt.contour(T_np, X_np, np.real(u_pred_np), levels=20, cmap='viridis')
plt.colorbar(label="Re(u)")
plt.xlabel("t")
plt.ylabel("x")
plt.title("Contour of Re(u)")

# 3D曲线图
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(T_np, X_np, np.real(u_pred_np), cmap='viridis')
ax.set_xlabel("t")
ax.set_ylabel("x")
ax.set_zlabel("Re(u)")

# 概率密度 |u|^2
plt.figure(figsize=(10, 6))
plt.pcolormesh(T_np, X_np, np.abs(u_pred_np)**2, shading='auto', cmap='viridis')
plt.colorbar(label="|u|^2")
plt.xlabel("t")
plt.ylabel("x")
plt.title("Probability density")
plt.show()
```

致谢

首先，我要由衷地感谢我的父母。正是他们无私的支持与付出，才让我能够坚持至今。这份恩情难以言表，唯有以不懈的努力回报！其次，我要特别感谢同门的师兄弟姐妹和挚友们。在漫长的申请季里，是你们的陪伴让我走出低谷，是你们及时的帮助解决了我的种种困境。没有你们的支持，我绝不可能走到今天，这份情谊我将永远铭记于心！我还要向所有指导过我的老师们致以最深的谢意。你们不仅授我以专业知识，更在我申请的关键时刻给予悉心指导和宝贵建议。对此，我始终心怀感激！最后，希望自己在接下来的 courant 所的求学之路上乘风破浪，学有所成！